

Sistemas Operativos en Tiempo Real

Codiseño HW/SW – Sistemas Integrados Digitales

Marcos Martínez Peiró mpeiro@eln.upv.es

*Máster Universitario de Ingeniería de Sistemas Electrónicos – MUISE –
Máster Universitario de Ingeniería de Telecomunicaciones – MUIT -*

Rtos –Real Time Operating Systems

Organización

- Sesión 1. Introducción.
 - Concepto de Sistema Embebido.
 - Arquitectura SW de un Sistema Embebido.
 - Concepto de Rtos y Principales Rtos.
- Sesión 2. Micro C
 - Inicialización sobre NIOS II.
 - Tareas.
 - Gestión de Tiempo.
 - Interrupciones.
 - Descripción Lab1.
- **Sesión 3. Micro C**
 - Comunicación entre tareas.
 - Gestión de Memoria.
 - Hooks.
 - Descripción Lab 2 y Lab 3

Sesión 3

Micro C (UCOS)

- Comunicación entre tareas.
- Gestión de Memoria.
- Timers.
- Hooks.
- Descripción Lab 2 y Lab 3

Micro C

Necesidades de Comunicación

- Habitualmente las tareas no son independientes, ya que se comunican o comparten recursos. Esto complica la planificación.
- **Compartir datos entre sensores y procesador**
 - Una tarea lee datos de un sensor (temperatura) y otra tarea los procesa para calcular promedios.
 - Necesidad: Transferir valores de la tarea de adquisición a la tarea de cálculo.
 - Mecanismo: **Cola de mensajes (Message Queue) o Mailbox.**
- **Señalización de eventos**
 - Una tarea controla un motor y otra supervisa varios eventos.
 - Necesidad:
Cuando se pierde aceite, se pulsa un interruptor, se sobrecalienta, ...la tarea del motor debe detenerse.
 - Mecanismo: **Flag de eventos.**
- **Sincronización de acceso a recursos compartidos**
 - Varias tareas escriben en la misma memoria o puerto serie.
 - Necesidad: Evitar condiciones de carrera y corrupción de datos.
 - Mecanismo: **Semáforo de exclusión mutua (Mutex).**

Micro C

Problemas de Comunicación

- Problemas de la comunicación entre tareas:
 - ¿Cómo pasar la información?
 - ¿Cómo asegurar que las tareas no se interfieran al efectuar actividades críticas?
 - ¿Cómo conseguir la secuencia correcta cuando existen dependencias?
- Problema de **inversión de prioridades**. Una tarea puede verse bloqueada por una de inferior prioridad.
- **Condición de carrera**: situación en la que dos o más tareas leen o escriben en un área compartida y el resultado final depende de los instantes de ejecución de cada una.
- Ejemplo: $X = X + 1$;
 - a) carga valor de X (memoria) en registro
 - b) incrementa valor en registro
 - c) almacena valor del registro en X (memoria)
- Son tres operaciones en las que dos tareas A y B pueden intercalarse dando lugar a resultados incorrectos. ASEGURAR ESCRITURAS/LECTURAS **INDIVISIBLES** (RTOS)

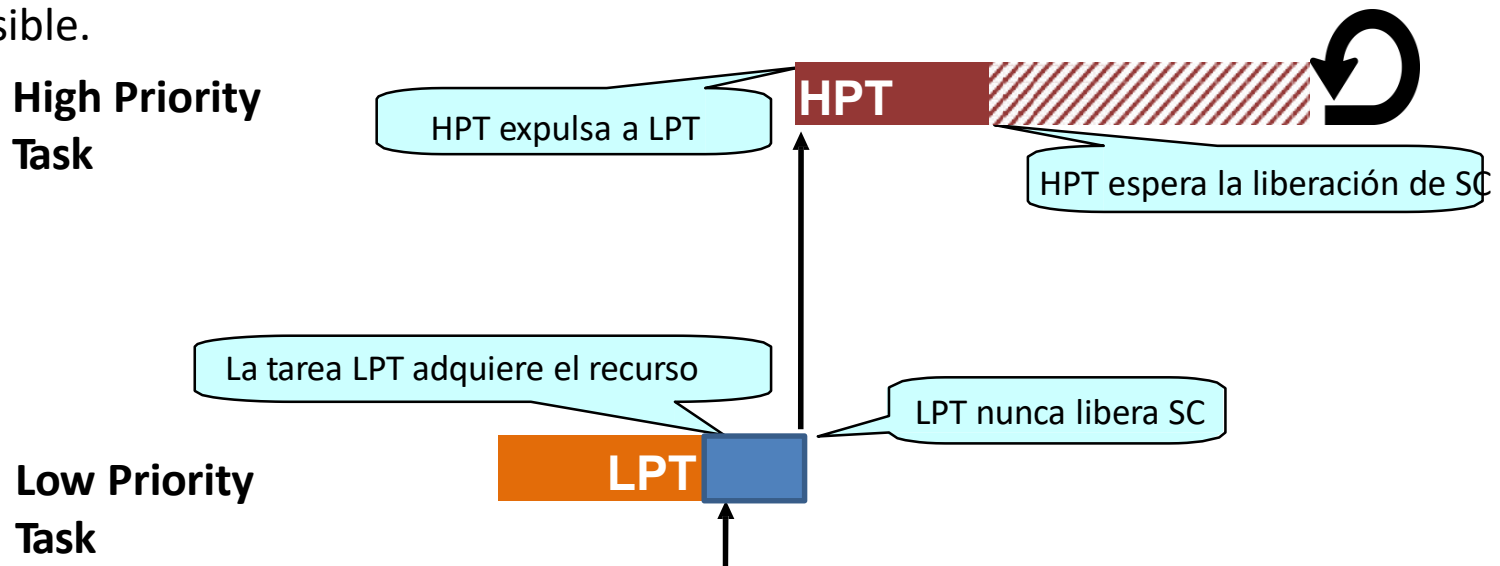
Problemas de Comunicación

- **Sección crítica (SC):** conjunto de operaciones que deben ejecutarse de manera indivisible.
- La protección de una sección crítica se consigue con **exclusión mutua**.
- Los mecanismos que garantizan exclusión mutua son primordiales en el diseño de los sistemas operativos.
- Las condiciones de carrera se evitarían si:
 - Que no haya en ningún momento dos tareas en sus SC
 - No se puede suponer nada acerca de velocidades o nº de CPUs
 - Ninguna tarea fuera de su SC puede bloquear a otras
 - Ninguna tarea podrá esperar indefinidamente para entrar en su SC

Micro C

Problemas de Comunicación

- **Sección crítica (SC):** conjunto de operaciones que deben ejecutarse de manera indivisible.



- **SOLUCION 1: Exclusión mutua con espera activa:** mientras un proceso está en la SC el otro espera.
- Prohibición de interrupciones: **OS_ENTER_CRITICAL()**, **OS_EXIT_CRITICAL()**. Típico en uCOSII.
 - la tarea deshabilita las interrupciones al entrar en la SC y las habilita al salir.
 - Se garantiza que no se conmuta a otra tarea.
 - Arriesgado dar a las tareas de usuario esa capacidad (colgar el sistema por error).
 - Es útil a nivel de SO pero no en tareas de usuario.

Problemas de Comunicación. Espera Activa.

- **SOLUCION 1: Exclusión mutua con espera activa:** mientras un proceso está en la SC el otro espera.
- A bajo nivel el kernel proporciona dos mecanismos:
 - Deshabilitar las interrupciones: `OS_ENTER_CRITICAL()` y `OS_EXIT_CRITICAL()`.
 - Debe usarse con mucha precaución, una llamada al sistema con las interrupciones deshabilitadas puede ser fatal. Por ejemplo, llamar a `OSTimeDly()` tras `OS_ENTER_CRITICAL()` colgará el sistema pues la ISR de reloj está desatendida.
 - Deshabilitar el planificador: `OSSchedLock()` y `OSSchedUnLock()`.
 - Las interrupciones permanecen habilitadas.
 - No debe llamarse ninguna función que fuerce a la tarea a suspenderse
 - porque nunca se planificará otra y el sistema se colgará.
 - De interés cuando una tarea de baja prioridad desea enviar mensajes a múltiples semáforos, buzones o colas y no se quiere que una tarea más prioritaria interfiera.

Micro C

Sección Crítica y Métodos de comunicación

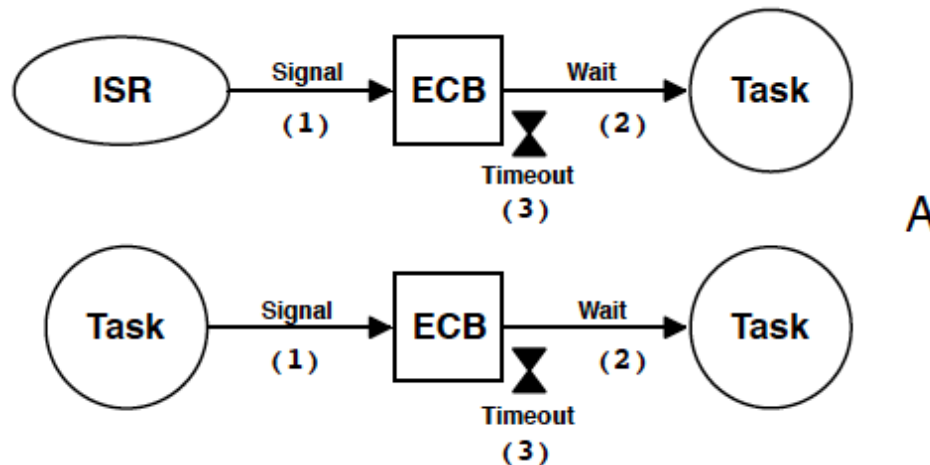
- SOLUCIÓN 2: **Exclusión mutua** sin **espera activa**: mientras un proceso está en la SC el otro es pasado a espera por el planificador (scheduler):
- **Único evento**:
 - Semáforos (variables compartidas protegidas)
 - Mailbox (paso de mensaje por punteros).
- **Grupos de eventos**:
 - Flag de eventos (semáforos) y Colas (conjunto de punteros mailbox)

Semáforos. Variable sobre la que se realizan dos operaciones: BAJAR y SUBIR.

- **BAJAR**
 - Si semáforo $\neq 0$, decrementa semáforo y sigue. Si semáforo = 0, se duerme.
 - Comprobar valor, decrementar y dormirse **se ejecutan de forma atómica**.
- **SUBIR**
 - Incrementa el semáforo, Si hay alguna tarea dormida en el semáforo el sistema elige una y le permite completar la operación BAJAR.
 - Incrementar y despertar también **se ejecutan de forma atómica**.

Semáforos en UCOS. Event Control Block.

- El kernel usa los ECBs para permitir interacción entre tareas con: **semáforos**, **banderas**, **buzones** o **colas de mensajes**.

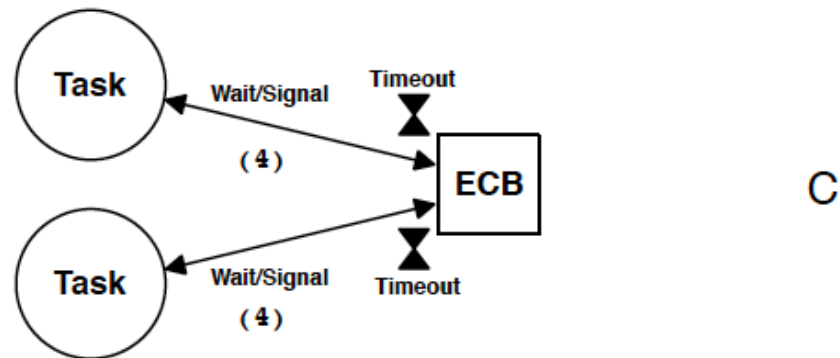
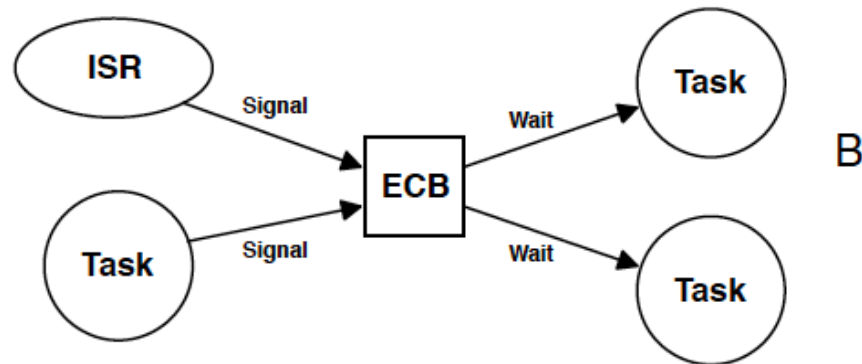


- Una ISR o una tarea pueden enviar una señal a un ECB (1).
- Solo una tarea puede esperar a otra tarea o a una ISR (2).
- Un timeout opcional se puede especificar para la tarea en espera (3).

Micro C

Semáforos en UCOS. Event Control Block.

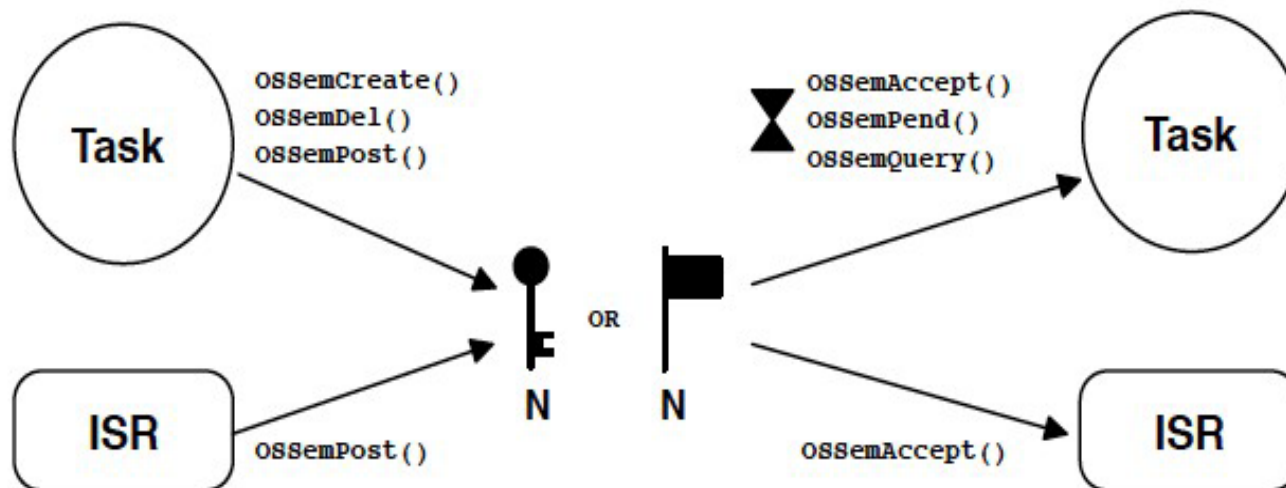
- Múltiples tareas pueden esperar una señal en un ECB.
- La más prioritaria la recibirá y se pondrá en *“ready”*.
- Cuando un ECB se usa como semáforo las tareas pueden esperar y enviar señales al ECB.



Micro C

Semáforos en UCOS. Event Control Block.

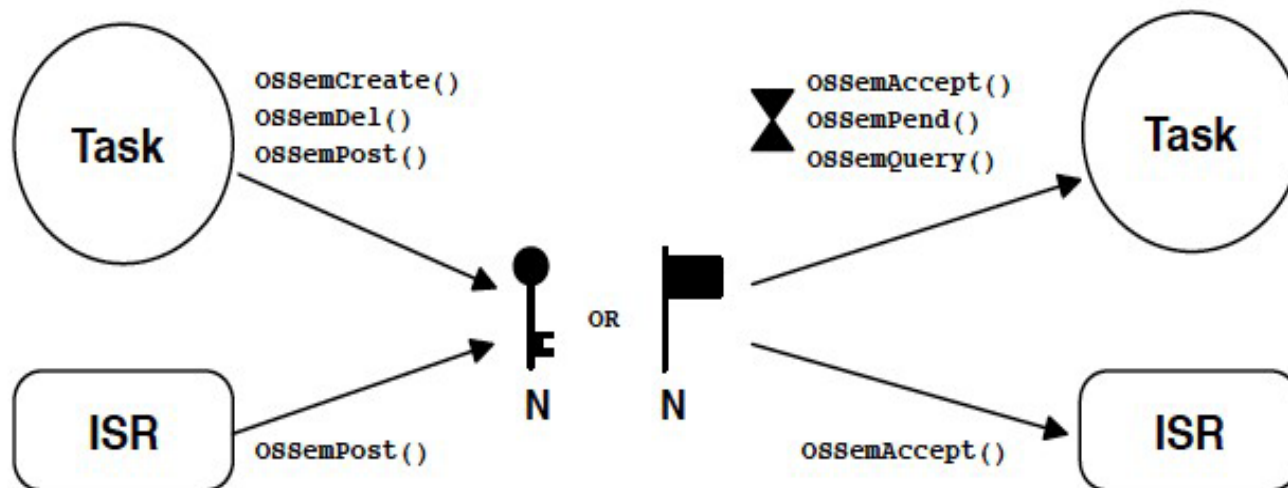
- Se implementan como un contador y una lista de tareas esperando en él.
- Los servicios deseados deben habilitarse en [OS_CFG.H](#) (o [BSP Editor](#)) para su compilación o no.
- La espera se puede limitar con timeouts para prevenir errores.



Micro C

Semáforos en UCOS. Event Control Block.

- Se implementan como un contador y una lista de tareas esperando en él.
- Los servicios deseados deben habilitarse en [OS_CFG.H](#) (o [BSP Editor](#)) para su compilación o no.
- La espera se puede limitar con timeouts para prevenir errores.



Micro C

Semáforos en UCOS. Event Control Block.

- **OS_EVENT *pevent;** declara un puntero a una estructura semáforo
- **pevent = OSSemCreate (INT16U cnt);**
 - crea al semáforo con un valor inicial (**cnt**), inicializa una lista de las tareas que esperan en el semáforo y apunta **pevent** a esa estructura.
- **void OSSemPend (OS_EVENT *pevent, INT16U timeout, INT8U *err)**
 - espera un cierto tiempo –timeout- (o para siempre si timeout=0) hasta que haya un recurso disponible-> **BAJAR semáforo**
- **INT8U OSSemPost(OS_EVENT *pevent)**
 - libera un recurso en un semaforo, con un posible cambio de contexto si una tarea de mayor prioridad estaba a la espera de ese recurso-> **SUBIR semáforo**

❖ **Practique con IA**

❖ **Revise el Manual de referencia**

Semáforos en UCOS. Event Control Block.

- **INT16U OSSemAccept (OS_EVENT *pevent)**
 - similar a **OSSemPend**, pero si no hay ningún recurso la tarea continúa sin suspenderse -> **BAJAR semáforo (sin espera)**
- **INT8U OSSemQuery (OS_EVENT *pevent, OS_SEM_DATA *pdata)**
 - obtiene información sobre el estado de un semáforo
- **OS_EVENT OSSemDel (OS_EVENT *pevent, INT8U opt, INT8U *err)**
 - elimina un semáforo. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

❖ **Practique con IA**

❖ **Revise el Manual de referencia**

```
#include "includes.h"

OS_EVENT *sem_recurso;

void Task1(void *pdata) {
    INT8U err;
    while (1) {
        OSSemPend(sem_recurso, 0, &err); // Espera recurso
        printf("Task1: usando recurso\n");
        OSTimeDlyHMSM(0, 0, 1, 0);      // Simula uso
        OSSemPost(sem_recurso);          // Libera recurso
    }
}

void Task2(void *pdata) {
    INT8U err;
    while (1) {
        OSSemPend(sem_recurso, 0, &err);
        printf("Task2: usando recurso\n");
        OSTimeDlyHMSM(0, 0, 1, 500);
        OSSemPost(sem_recurso);
    }
}

int main(void) {
    OSInit();
    sem_recurso = OSSemCreate(3); // Semáforo nivel 3
    OSTaskCreate(Task1, NULL, &Task1Stk[STACKSIZE-1], 5);
    OSTaskCreate(Task2, NULL, &Task2Stk[STACKSIZE-1], 6);
    OSStart();
    return 0;
}
```


Semáforos en UCOS. Event Control Block.

- **INT16U OSSemAccept (OS_EVENT *pevent)**
 - similar a **OSSemPend**, pero si no hay ningún recurso la tarea continúa sin suspenderse -> **BAJAR semáforo (sin espera)**
- **INT8U OSSemQuery (OS_EVENT *pevent, OS_SEM_DATA *pdata)**
 - obtiene información sobre el estado de un semáforo
- **OS_EVENT OSSemDel (OS_EVENT *pevent, INT8U opt, INT8U *err)**
 - elimina un semáforo. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

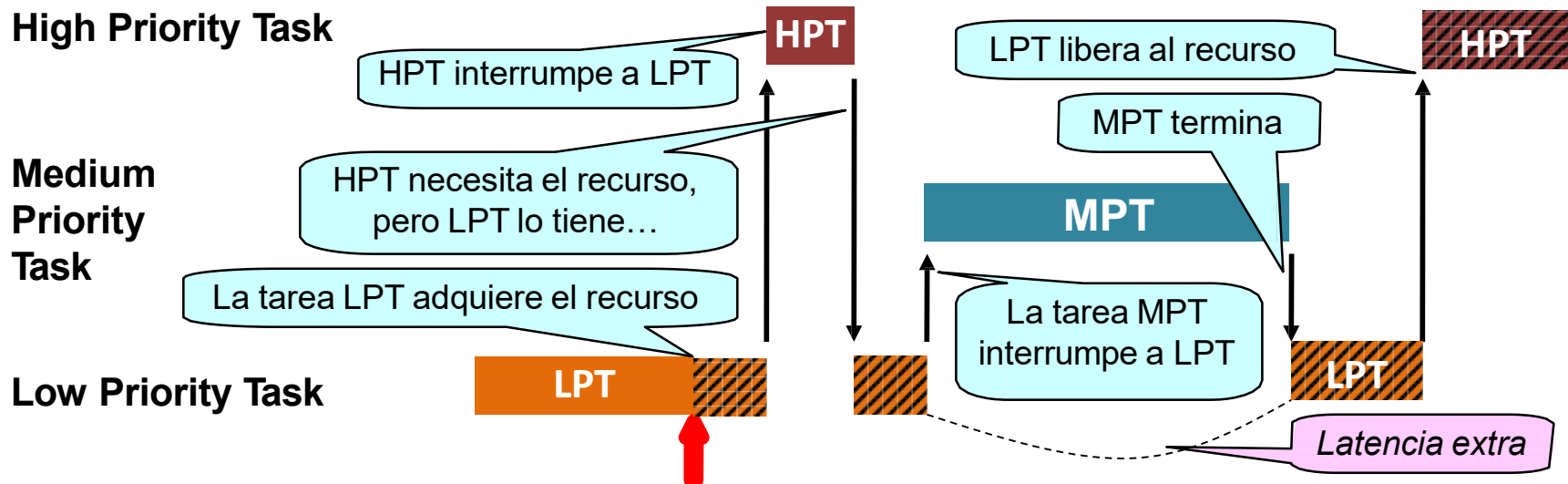
❖ **Practique con IA**

❖ **Revise el Manual de referencia**

Micro C

Semáforos en UCOS. Inversión de Prioridades.

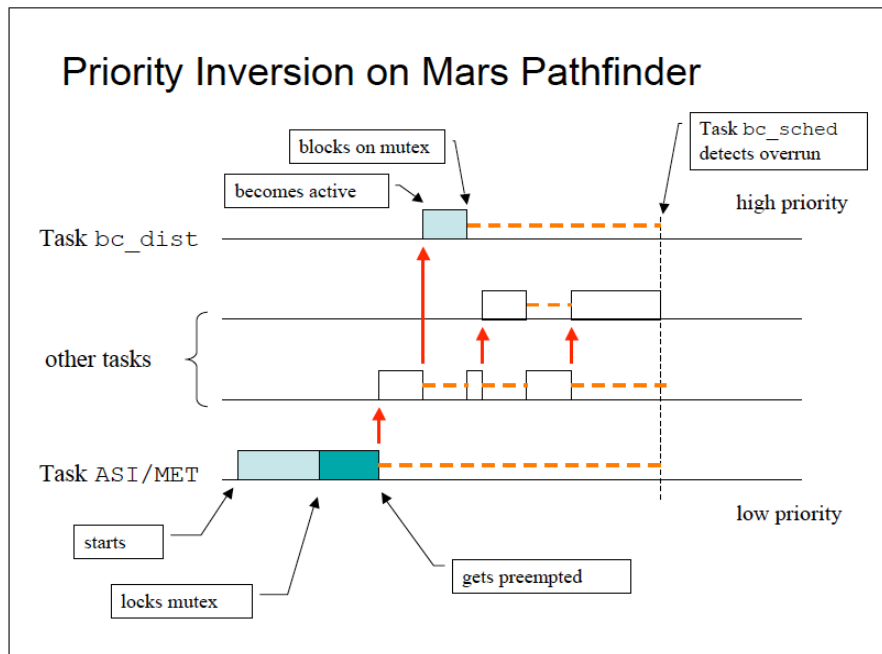
- La espera de un recurso bajo control de una tarea de menos prioridad puede demorar el tiempo de ejecución de tareas de más prioridad (lo que es correcto)
- Pero la tarea de mayor prioridad puede a su vez ser demorada por tareas de prioridad intermedia (se puede escapar a nuestro control)



Micro C

Semáforos en UCOS. Inversión de Prioridades.

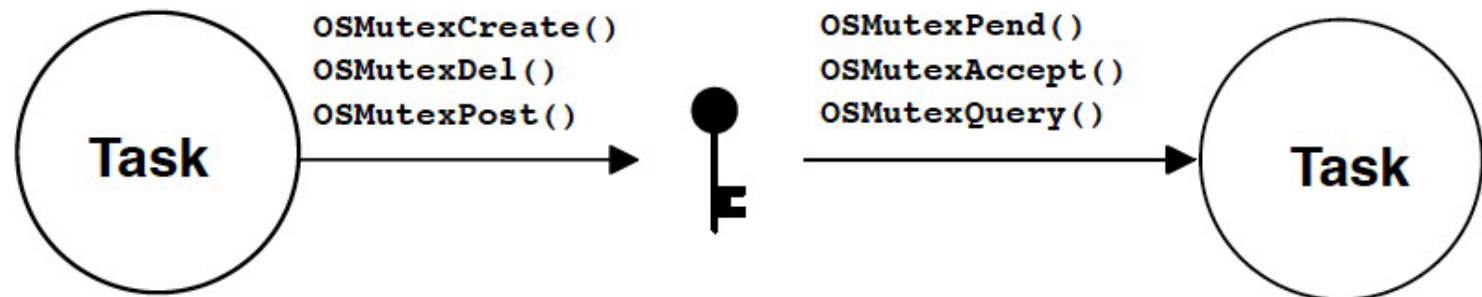
- Mars PathFinder. Clásico ejemplo de inversión de prioridades
- Aterrizó en Marte el 4 de julio de 1997
- “...experiences software glitches...”
- Sufría Resets cuando se dedicaba a recoger datos meteorológicos



Micro C

Semáforos en UCOS. Inversión de Prioridades.

- Solución: MUTEX con herencia de prioridades.
- Cuando una tarea adquiere el recurso su prioridad cambia de forma dinámica adquiriendo la PRIO mayor de todas las tareas que compiten por el recurso. Evita ser bloqueada.
- Los MUTEX son semáforos binarios que sirven para minimizar el problema de la inversión de prioridades. Ese proceso es administrado por el SO y no por el usuario. Se reserva un nivel de prioridad mayor que la máxima prioridad de las tareas que comparten el recurso (PIP: Priority Inheritance Priority).



Micro C

Semáforos en UCOS. Inversión de Prioridades.

- MUTEX con herencia de prioridades.

OS_EVENT *pevent;

- declara un puntero a una estructura mutex

pevent = OSMutexCreate (INT8U prio, INT8U *err);

- crea al *mutex* con un valor 1, le asigna la PIP, inicializa una lista de las tareas que esperan en el semáforo

void OSMutexPend (OS_EVENT *pevent, INT16U timeout, INT8U *err)

- espera un cierto tiempo (o por siempre) hasta que haya un recurso disponible
-> BAJAR semáforo

INT8U OSMutexPost(OS_EVENT *pevent)

- libera un recurso en un *mutex*, con un posible cambio de contexto si una tarea de mayor prioridad estaba a la espera de ese recurso
-> SUBIR semáforo

Micro C

Semáforos en UCOS. Inversión de Prioridades.

- MUTEX con herencia de prioridades.

INT16U OSMutexAccept (OS_EVENT *pevent, INT8U *err)

- similar a **OSMutexPend**, pero si no hay ningún recurso la tarea continúa sin suspenderse

INT8U OSMutexQuery (OS_EVENT *pevent, OS_MUTEX_DATA *pdata)

- obtiene información sobre el estado de un *mutex*

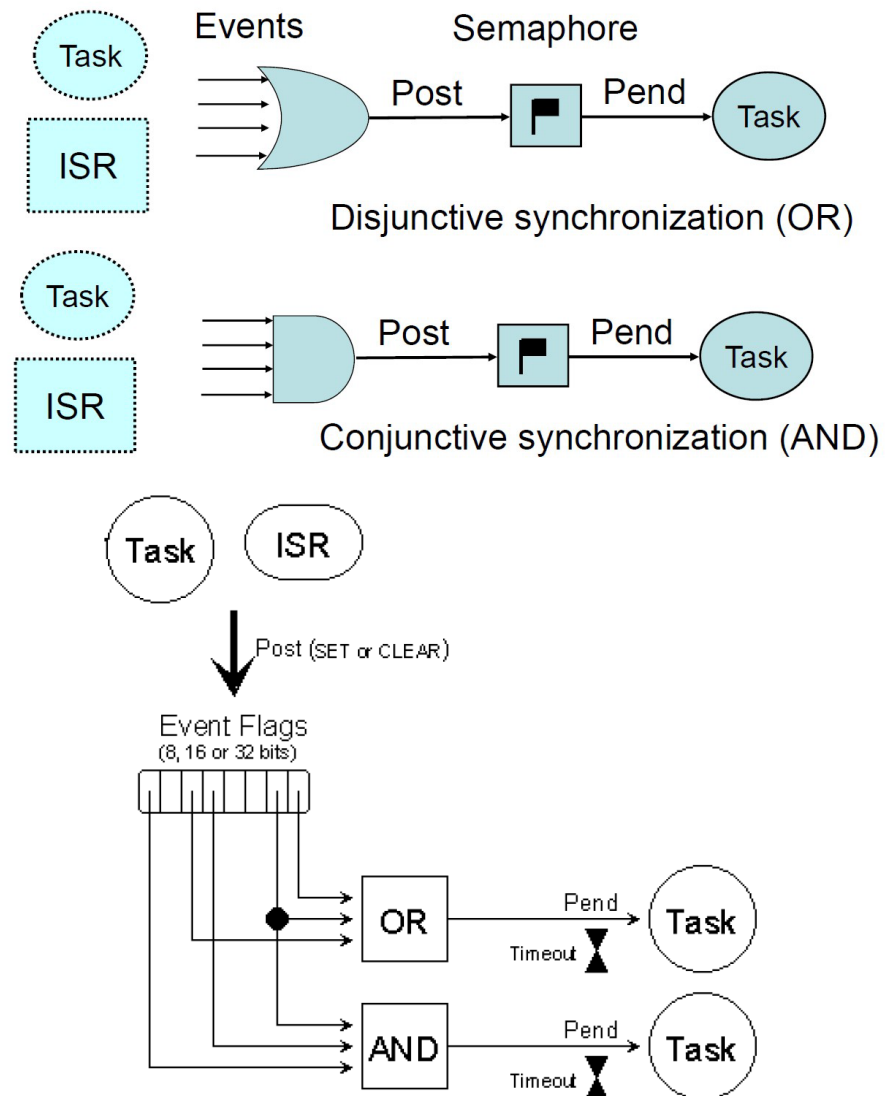
OS_EVENT *OSMutexDel (OS_EVENT *pevent, INT8U opt, INT8U *err)

- elimina un semáforo. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

Micro C

Event Flags en UCOS. Banderas de eventos.

- Una bandera se usa cuando una tarea debe sincronizarse con **múltiples** eventos.
- La sincronización puede ser disyuntiva (OR) o conjuntiva (AND).
- Tanto las tareas como las ISR pueden poner a “0” o “1” un flag, pero solo las tareas pueden quedar a las espera.
- Los servicios deseados deben habilitarse en `OS_CFG.H` para su compilación o no.



Micro C

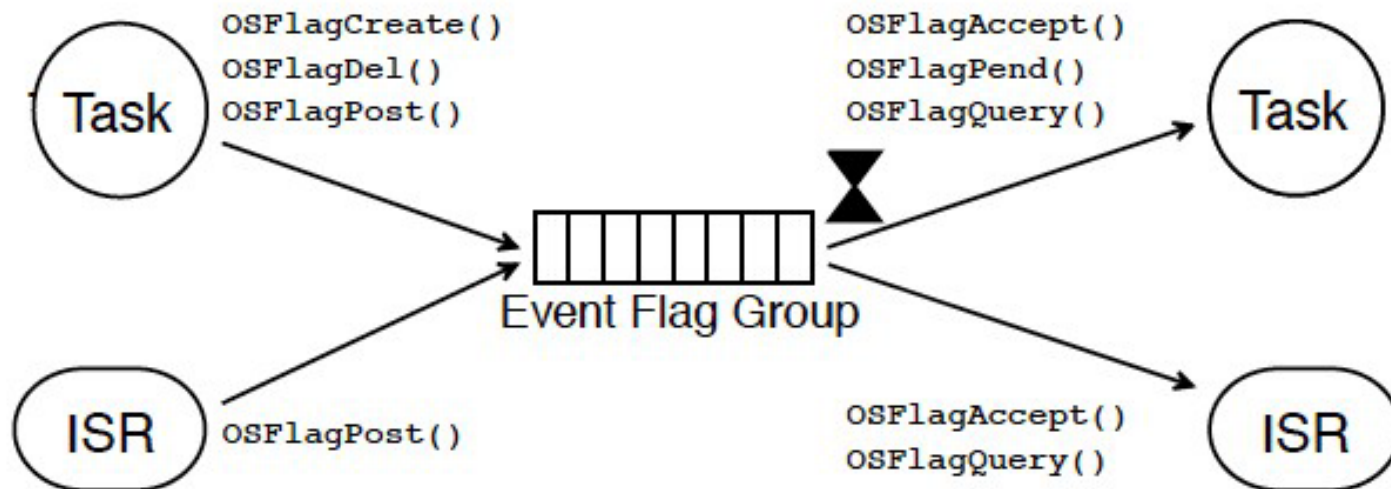
Event Flags en UCOS. Banderas de eventos.

`typedef INT8U (o INT16U o INT32U) OS_FLAGS;`

- al definir un objeto de este tipo se puede inicializar. Contiene un patrón de bits a “0” o “1”

`typedef struct {...} OS_FLAG_GRP;`

- define un grupo de flags y la lista de tareas esperando sobre esos eventos



Micro C

Event Flags en UCOS. Banderas de eventos.

`OS_FLAG_GRP *OSFlagCreate (OS_FLAGS flags, INT8U *perr)`

- crea una estructura `OS_FLAGS_GRP` lista para su uso posterior

`OS_FLAGS OSFlagPend (OS_FLAG_GRP *pgrp, OS_FLAGS flags, INT8U wait_type, INT16U timeout, INT8U *perr)`

- la tarea espera a que se cumpla una combinación de condiciones (bits a “0” o “1”) de acuerdo con `wait_type` (`OS_FLAG_WAIT_SET_ALL`, `OS_FLAG_WAIT_SET_ANY`, `OS_FLAG_WAIT_CLR_ALL`, `OS_FLAG_WAIT_CLR_ANY`) o vence el timeout

`OS_FLAGS OSFlagPost (OS_FLAG_GRP *pgrp, OS_FLAGS flags, INT8U opt, INT8U *perr)`

- según el valor de `opt` (`OS_FLAG_CLR`, `OS_FLAG_SET`) pone en ‘0’ o ‘1’ los flags indicados por `flags` en el grupo de flags `pgrp`.

Micro C

Event Flags en UCOS. Banderas de eventos.

```
#include "includes.h"

OS_FLAG_GRP *EventFlags;

void TaskWait(void *pdata) {
    INT8U err;
    OS_FLAGS flags_needed = (1 << 0) | (1 << 1) | (1 << 5); // Bits 0,1,5
    while (1) {
        OS_FLAGS result = OSFlagPend(EventFlags,
                                     flags_needed,
                                     OS_FLAG_WAIT_SET_ALL + OS_FLAG_CONSUME,
                                     100, // Timeout en ticks
                                     &err);

        if (err == OS_NO_ERR) {
            printf("TaskWait: condición cumplida, acción realizada\n");
        } else if (err == OS_TIMEOUT) {
            printf("TaskWait: timeout esperando flags\n");
        }
    }
}
```

```
void TaskSetBits(void *pdata) {
    INT8U err;
    while (1) {
        OSTimeDlyHMSM(0, 0, 1, 0);
        OSFlagPost(EventFlags, (1 << 0), OS_FLAG_SET, &err);
        OSTimeDlyHMSM(0, 0, 1, 0);
        OSFlagPost(EventFlags, (1 << 1), OS_FLAG_SET, &err);
        OSTimeDlyHMSM(0, 0, 1, 0);
        OSFlagPost(EventFlags, (1 << 5), OS_FLAG_SET, &err);
    }
}
```

```
int main(void) {
    OSInit();
    EventFlags = OSFlagCreate(0x00, &err); // Inicializa flags en 0
    OSTaskCreate(TaskWait, NULL, &TaskWaitStk[STACKSIZE-1], 5);
    OSTaskCreate(TaskSetBits, NULL, &TaskSetBitsStk[STACKSIZE-1], 6);
    OSStart();
    return 0;
}
```

Micro C

Event Flags en UCOS. Banderas de eventos.

OSFlagAccept (OS_FLAG_GRP *pgrp, OS_FLAGS flags, INT8U wait_type, INT8U *perr)

- idéntica a OSFlagPend pero no bloqueante

OSFlagQuery (OS_FLAG_GRP *pgrp, INT8U *perr)

- consulta el valor de los flags en un grupo

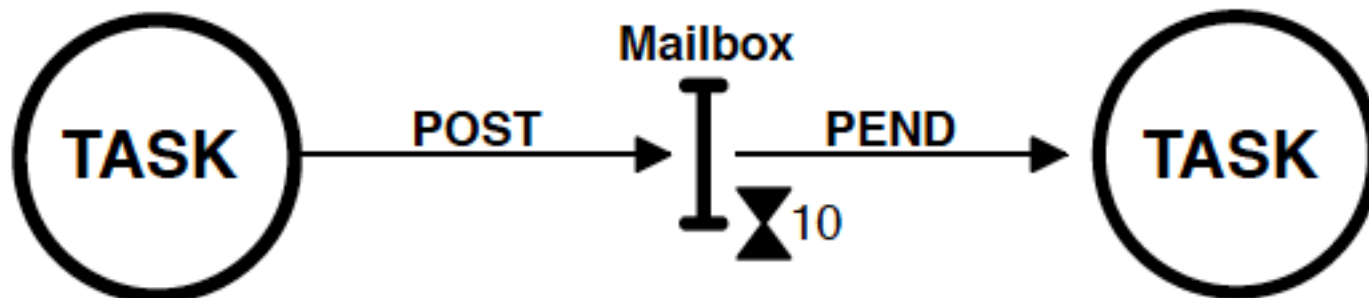
OSFlagDel (OS_FLAG_GRP *pgrp, INT8U opt, INT8U *perr)

- elimina un objeto OS_FLAGS_GRP. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

Micro C

MailBox en UCOS. Buzón de Mensajes.

- Un buzón de mensajes es un objeto que permite que una tarea o ISR envíe una variable tipo “puntero” a otra tarea, donde ese puntero típicamente apunta a una estructura propia de la aplicación, que contiene un **mensaje**.
- No se controla tan solo el acceso a un recurso, como con semáforos, sino que se envían datos complejos a una zona de memoria compartida apuntada por el puntero.
- Cada buzón tiene una lista asociada de tareas en espera. Si la lista está vacía la tarea se suspende hasta que se recibe un mensaje.



Micro C

MailBox en UCOS. Buzón de Mensajes.

`OS_EVENT *OSMboxCreate (void *msg)`

- crea el buzón especificando el valor inicial del puntero (habitualmente NULL)

`void *OSMboxPend (OS_EVENT * pevent, INT16U timeout, INT8U *err)`

- espera a que un mensaje llegue al buzón (o venza el timeout), devuelve un puntero al mensaje y borra el puntero original

`INT8U OSMboxPost (OS_EVENT * pevent, void *msg)`

- deposita un mensaje en el buzón que pondrá “ready” la tarea más prioritaria esperando en el mismo

`INT8U OSMboxPostOpt (OS_EVENT * pevent, void *msg, INT8U opt)`

- deposita un mensaje en el buzón para todas las tareas esperando en el buzón

`void *OSMboxAccept (OS_EVENT * pevent)`

- idéntica a `OSMboxPend` pero no bloqueante

`INT8U OSMboxQuery (OS_EVENT * pevent, OS_MBOX_DATA *pdata)`

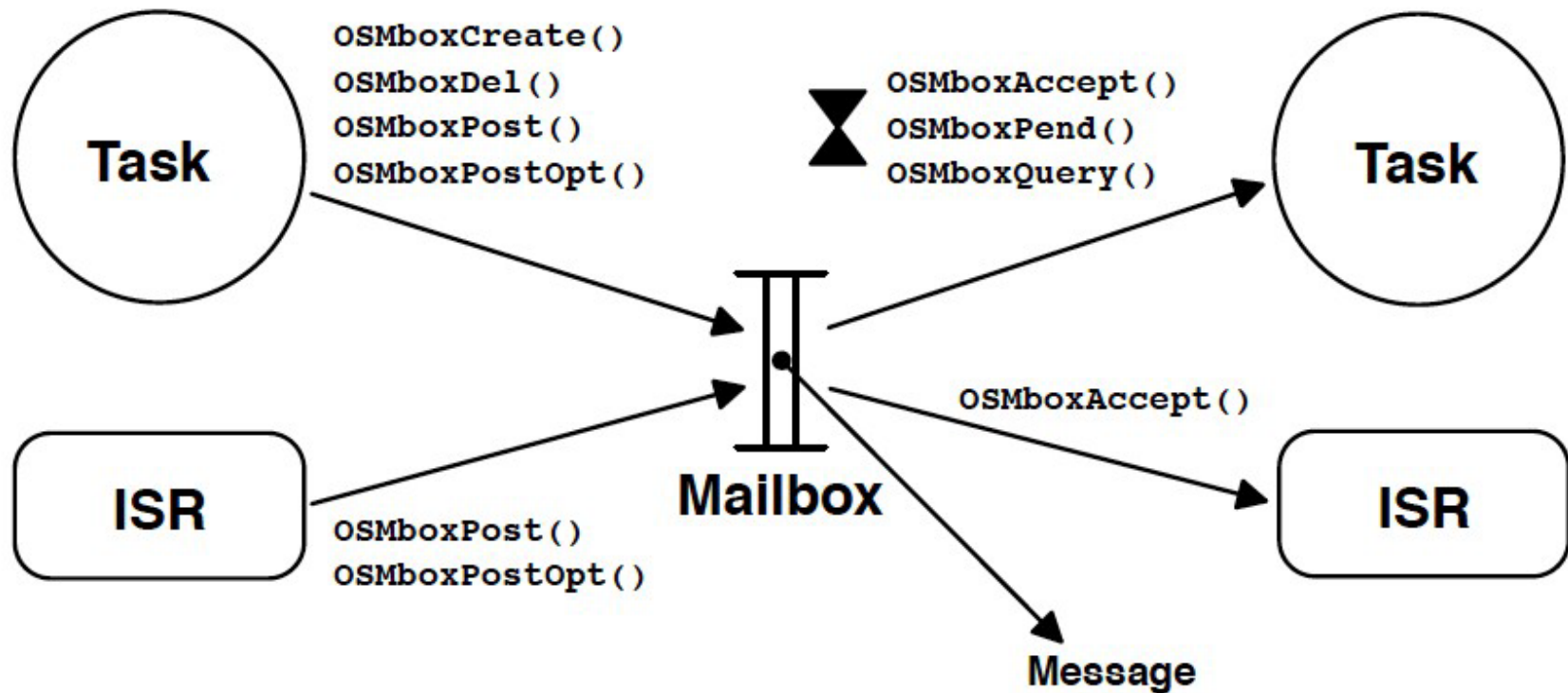
- consulta el evento asociado al buzón

`OS_EVENT *OSMboxDel (OS_EVENT * pevent, INT8U opt, INT8U *err)`

- elimina el evento asociado al buzón. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

Micro C

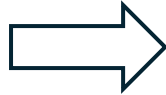
MailBox en UCOS. Buzón de Mensajes.



Micro C

MailBox en UCOS.

MailBox compartido



adc_value local



*received_value local



Las tareas solo comparten el MailBox. Sus variables y punteros de proceso son locales. Elimina secciones críticas.

```
#include "includes.h"

#define TASK_STACK_SIZE 256
#define TASK_ADC_PRIIO 5
#define TASK_PROC_PRIIO 6
#define ADC_THRESHOLD 800
#define ADC_OFFSET 10

OS_STK TaskADCStk[TASK_STACK_SIZE];
OS_STK TaskProcStk[TASK_STACK_SIZE];

OS_EVENT *MailboxADC;

void TaskADC(void *pdata) {
    INT16U adc_value;
    while (1) {
        // Simulación de lectura ADC
        adc_value = rand() % 1024; // Valor entre 0 y 1023
        OSMboxPost(MailboxADC, (void *)&adc_value);
        OSTimeDlyHMSM(0, 0, 1, 0); // Espera 1 segundo
    }
}

void TaskProcessor(void *pdata) {
    INT16U *received_value;
    INT16U processed_value;
    while (1) {
        received_value = (INT16U *)OSMboxPend(MailboxADC, 0, NULL);
        processed_value = *received_value - ADC_OFFSET;

        if (processed_value > ADC_THRESHOLD) {
            printf("ALERTA: Valor %d supera umbral\n", processed_value);
        } else {
            printf("Valor procesado: %d\n", processed_value);
        }
    }
}
```

Micro C

MailBox en UCOS. Buzón de Mensajes.

■ Programa principal.

```
int main(void) {
    OSInit();

    MailboxADC = OSMboxCreate(NULL);

    OSTaskCreate(TaskADC, (void *)0, &TaskADCStk[TASK_STACK_SIZE - 1], TASK_ADC_PRI0);
    OSTaskCreate(TaskProcessor, (void *)0, &TaskProcStk[TASK_STACK_SIZE - 1], TASK_PROC_PRI0);

    OSStart();
    return 0;
}
```


Micro C

MailBox en UCOS. Envío con y sin Broadcast.

```
void task1(void* pdata)
{
    INT8U error_code = OS_NO_ERR;
    int t1;

    while (1)
    {
        t1 = OSTimeGet();
        OSSemPend(shared_jtag_sem,0,&error_code);
        printf("Task1 sending message: %d ms\n",t1);
        OSSemPost(shared_jtag_sem);
        //Post the message with broadcast to all pending tasks
        error_code = OSMBboxPostOpt(MSG_box,(void *)&t1,OS_POST_OPT_BROADCAST);
        OSTimeDlyHMSM(0, 0, 1, 0);
    }
}
```

Sending task

```
void task2(void* pdata)
{
    INT8U error_code = OS_NO_ERR;
    int t1;
    int *msg_rx;

    while (1)
    {
        //Pend messages sent from task1
        msg_rx = (int*)OSMBboxPend(MSG_box,0,&error_code);
        t1 = OSTimeGet();

        OSSemPend(shared_jtag_sem,0,&error_code);
        printf("Task2 received message: %d ms (at %d ms)\n",*msg_rx ,t1);
        OSSemPost(shared_jtag_sem);
    }
}
```

Receiving task

(Similar for task3)

Running application with broadcast

```
Task1 sending message: 1071 ms
Task2 received message: 1071 ms (at 1076 ms)
Task3 received message: 1071 ms (at 1079 ms)
Task1 sending message: 2076 ms
Task2 received message: 2076 ms (at 2081 ms)
Task3 received message: 2076 ms (at 2083 ms)
```

Running application w/o broadcast

```
Task1 sending message: 1097 ms
Task2 received message: 1097 ms (at 1100 ms)
Task1 sending message: 2100 ms
Task2 received message: 2100 ms (at 2104 ms)
```

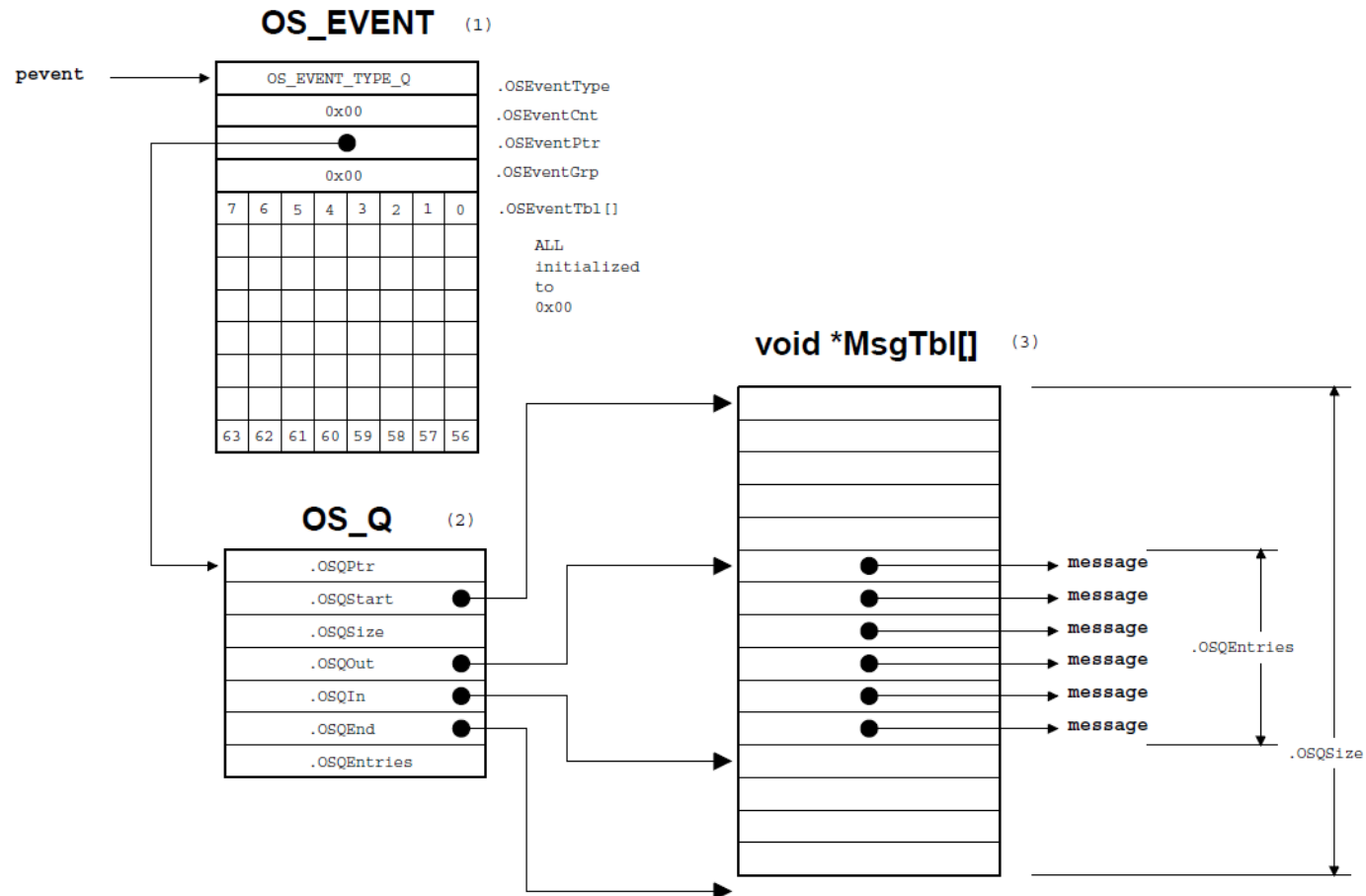
```
error_code = OSMBboxPost(MSG_box,(void *)&t1);
```

Queues en UCOS. Colas de Mensajes.

- Una cola de mensajes es básicamente un vector de mailbox.
- Una tarea (o una ISR) puede depositar un mensaje (el puntero) en la cola.
- Una o varias tareas pueden recibir el mensaje (no las ISRs).
- Los mensajes se pueden extraer en orden FIFO o LIFO.
- Cada cola tiene una lista asociada de tareas en espera. Si la lista está vacía la tarea se suspende hasta que se recibe un mensaje.

Queues en UCOS. Colas de Mensajes.

- Estructura interna de la cola en UCOSII.



Micro C

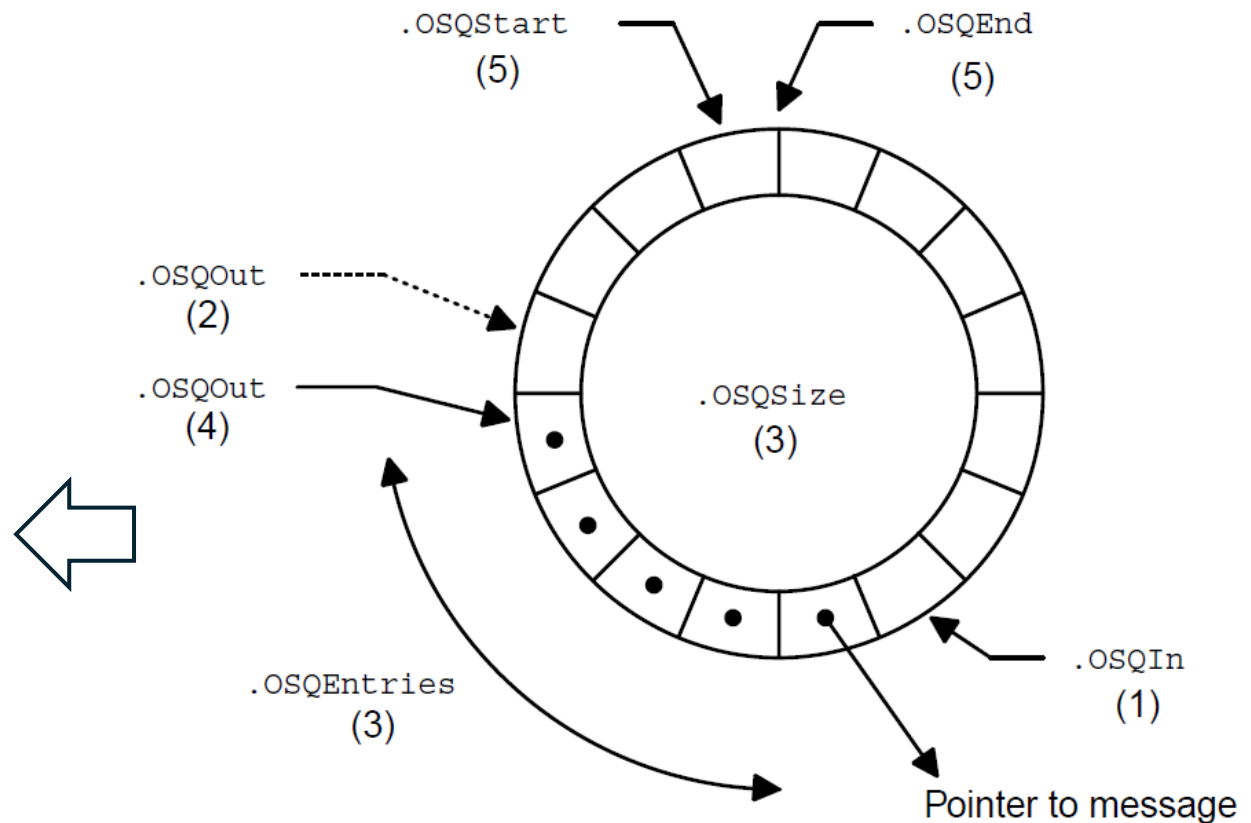
Queues en UCOS. Colas de Mensajes.

- Se puede interpretar como un buffer circular de mensajes

Muy útil para:

1. Tratar mensajes que vengan de UARTs o dispositivos productores (ADCs).

2. Para sincronizar tareas productoras con consumidoras a distintas velocidades



Micro C

Queues en UCOS. Colas de Mensajes.

`OS_EVENT *OSQCreate (void **start, INT16U size)`

- crea la cola, `start` es el puntero al array de mensajes, `size` es el tamaño del array

`void *OSQPend (OS_EVENT * pevent, INT16U timeout, INT8U *err)`

- espera a que un mensaje llegue a la cola (o venza el timeout) y devuelve un puntero al mensaje

`INT8U OSQPost (OS_EVENT * pevent, void *msg)`

- deposita un mensaje en la cola (FIFO) que pondrá “ready” la tarea más prioritaria esperando

`INT8U OSQPostFront (OS_EVENT * pevent, void *msg)`

- deposita un mensaje en cabeza de la cola (LIFO), pondrá “ready” la tarea más prioritaria esperando

`INT8U OSQPostOpt (OS_EVENT * pevent, void *msg, INT8U opt)`

- deposita un mensaje en el buzón para todas las tareas esperando en la cola (FIFO o LIFO)

Micro C

Queues en UCOS. Colas de Mensajes.

`void *OSQAccept (OS_EVENT * pevent)`

- idéntica a `OSQPend` pero no bloqueante

`INT8U OSQFlush (OS_EVENT * pevent)`

- borra todos los mensajes en una cola

`INT8U OSQQuery (OS_EVENT * pevent, OS_Q_DATA *pdata)`

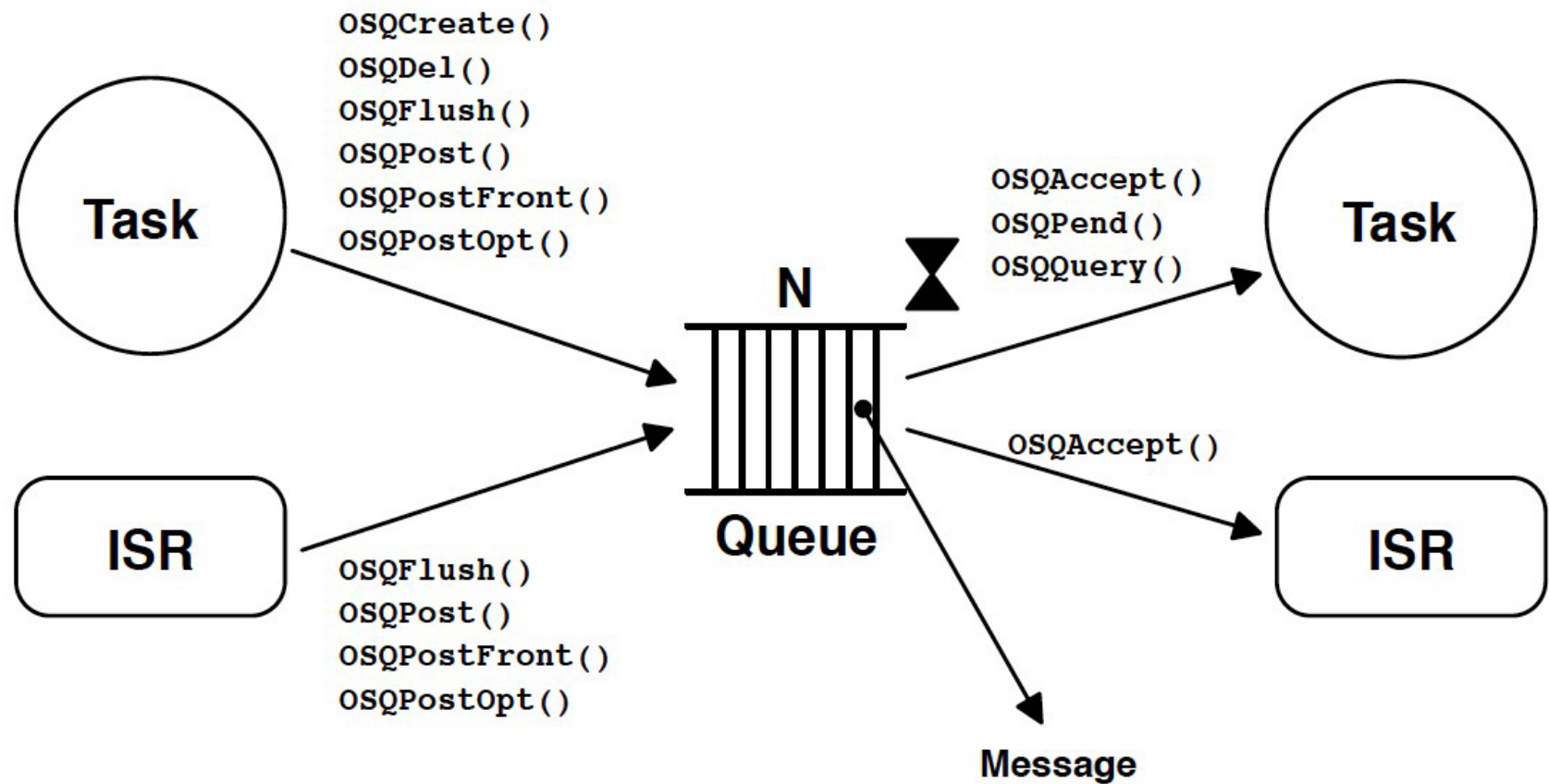
- consulta el evento asociado a la cola

`OS_EVENT *OSQDel (OS_EVENT * pevent, INT8U opt, INT8U *err)`

- elimina el evento asociado a la cola. Precaución: solo hacerlo cuando se han eliminado todas las tareas que lo usan

Micro C

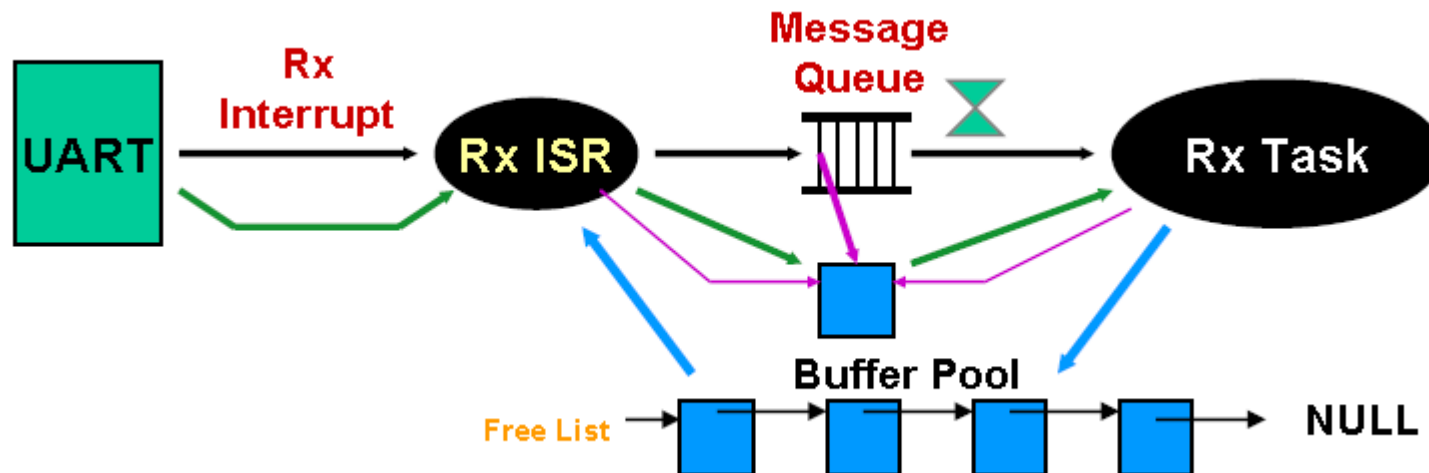
Queues en UCOS. Colas de Mensajes.



Micro C

Queues en UCOS. Colas de Mensajes.

■ Ejemplo UART



- **TaskADC:**

Lee datos (simulados) del ADC y rellena una estructura con:
Temperatura - Canal - ruido

Envía un puntero a esa estructura a una cola de mensajes.

- **TaskProcessor:**

Recibe el puntero desde la cola y procesa todos los campos
(por ejemplo, corrige ruido, verifica umbral de temperatura).

Micro C

Queues en UCOS. Colas de Mensajes.

```
#include "includes.h"
#include <stdlib.h>
#include <stdio.h>

#define TASK_STACK_SIZE 256
#define TASK_ADC_PRI0 5
#define TASK_PROC_PRI0 6
#define QUEUE_SIZE 10
#define TEMP_THRESHOLD 75.0

typedef struct {
    float temperatura;
    INT8U canal;
    float ruido;
} ADC_Data;

OS_STK TaskADCStk[TASK_STACK_SIZE];
OS_STK TaskProcStk[TASK_STACK_SIZE];

OS_EVENT *MsgQueue;
void *QueueStorage[QUEUE_SIZE];
```

```
int main(void) {
    OSInit();

    MsgQueue = OSQCreate(&QueueStorage[0], QUEUE_SIZE);

    OSTaskCreate(TaskADC, NULL, &TaskADCStk[TASK_STACK_SIZE - 1],
TASK_ADC_PRI0);
    OSTaskCreate(TaskProcessor, NULL, &TaskProcStk[TASK_STACK_SIZE - 1],
TASK_PROC_PRI0);

    OSStart();
    return 0;
}
```

Micro C

Queues en UCOS. Colas de Mensajes.

```
void TaskADC(void *pdata) {
    ADC_Data *adc_msg;
    while (1) {
        // Reservar memoria para el mensaje
        adc_msg = (ADC_Data *)malloc(sizeof(ADC_Data));
        if (adc_msg != NULL) {
            // Simulación de lectura ADC
            adc_msg->temperatura = (rand() % 100); // 0 a 99 grados
            adc_msg->canal = rand() % 4;           // Canal 0-3
            adc_msg->ruido = ((float)(rand() % 10)) / 10.0; // Ruido 0.0 a 0.9

            // Enviar puntero a la cola
            if (OSQPost(MsgQueue, (void *)adc_msg) == OS_NO_ERR) {
                printf("ADC: enviado canal %d, temp %.2f, ruido %.2f\n",
                    adc_msg->canal, adc_msg->temperatura, adc_msg->ruido);
            } else {
                free(adc_msg); // Liberar si no se pudo enviar
            }
        }
        OSTimeDlyHMSM(0, 0, 1, 0); // Cada 1 segundo
    }
}
```

Micro C

Queues en UCOS. Colas de Mensajes.

```
void TaskProcessor(void *pdata) {
    ADC_Data *received;
    INT8U err;
    while (1) {
        received = (ADC_Data *)OSQPend(MsgQueue, 0, &err);
        if (err == OS_NO_ERR && received != NULL) {
            // Procesamiento: eliminar ruido y verificar umbral
            float valor_filtrado = received->temperatura - received->ruido;
            if (valor_filtrado > TEMP_THRESHOLD) {
                printf("ALERTA: Canal %d supera umbral (%.2f)\n",
                    received->canal, valor_filtrado);
            } else {
                printf("Procesado: Canal %d, Temp %.2f\n",
                    received->canal, valor_filtrado);
            }
            free(received); // Liberar memoria después de procesar
        }
    }
}
```

- **Cola almacena punteros**, no datos directos → eficiente para estructuras grandes.
- **malloc/free** para gestionar memoria dinámica (en sistemas embebidos se recomienda usar un pool de memoria).
- **Procesamiento completo**: elimina ruido y verifica umbral.
- **Liberación de memoria** tras procesar para evitar fugas.

Micro C

Gestión de Memoria

- La programación en C no permite asegurar un espacio de memoria seguro para los datos en tiempo de ejecución; por ejemplo, el uso habitual de punteros puede llevar al programador a escribir un volumen elevado de información en la dirección apuntada y, en caso de existir datos previos importantes en esa zona de memoria, podrían ser aplastados por los nuevos valores.
- En ANSI C, el uso de reserva de memoria mediante `malloc()`, `calloc()`, `free()` es habitual si queremos reservar un espacio de memoria para trabajar con datos. Sin embargo, salvo que la memoria sea muy grande (cosa que no ocurre en embebidos), estas funciones no son seguras y pueden generar desbordamientos de uso de memoria al no tener elementos de protección.
- La fragmentación de memoria debida a su uso continuo en tiempo de ejecución puede llevar a que no se pueda obtener el espacio solicitado o que los tiempos de ejecución de las llamadas a estas funciones sean muy elevados debido a la búsqueda de espacio libre en memoria para la reserva.

Micro C

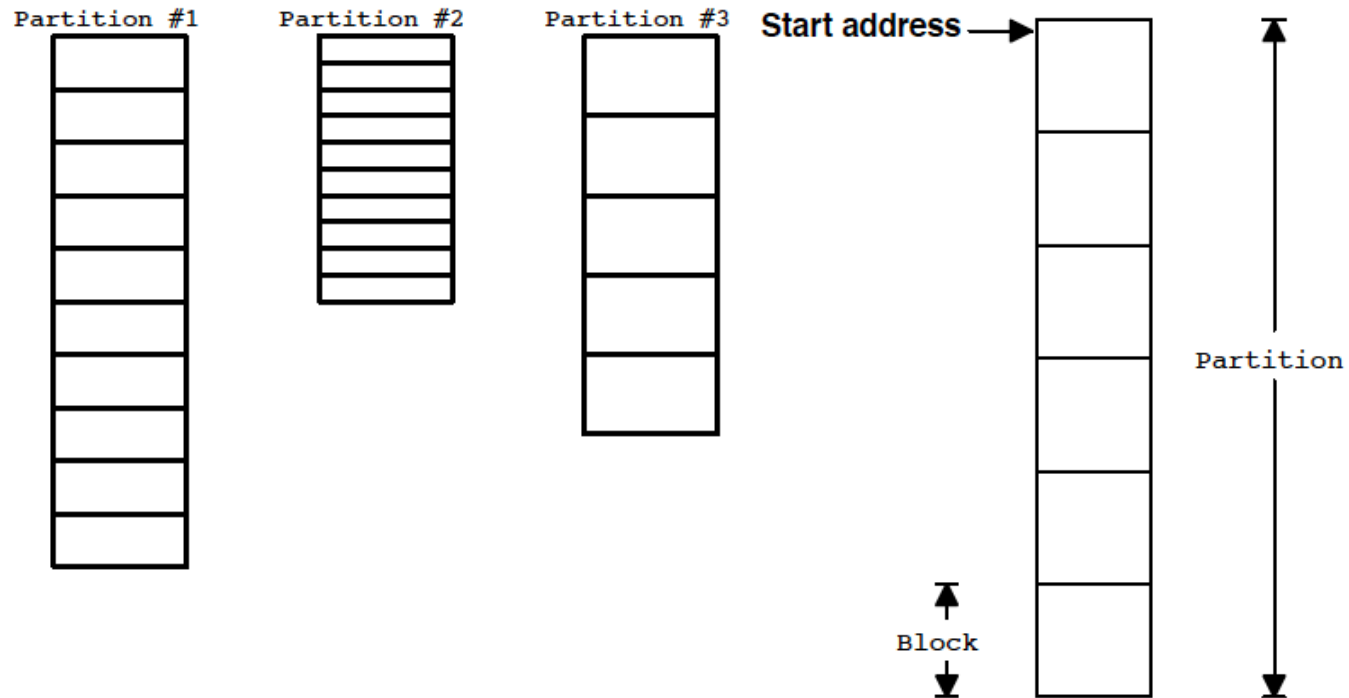
Gestión de Memoria

- La fragmentación de la memoria requiere tareas periódicas de mantenimiento que reagrupen los bloques (*GC: garbage collection*), y que al igual que *malloc()* tienen duración no determinista.
- En un RTOS esta latencia indefinida es inaceptable, por lo que en uC/OS II la memoria se divide en **particiones**.
- Cada partición contiene una cantidad entera de bloques de **tamaño fijo**.
- Las tareas deben pedir (**GET**) y devolver (**PUT**) bloques de la partición.
- La asignación la realiza uCOS en tiempo constante y es determinista.
- Si una tarea pide varios bloques de un tamaño dado no pueden hacerse presunciones sobre su posición física, por lo que deben ser tratados como unidades no relacionadas.
- Los servicios deseados (**OSMemCreate()**, **OSMemGet()**, **OSMemPut()** y **OSMemQuery()**), deben habilitarse en **OS_CFG.H** para su compilación o no.

Micro C

Gestión de Memoria

- uC/OS II emplea una estructura llamada Memory Control Block (MCB) para administrar la memoria



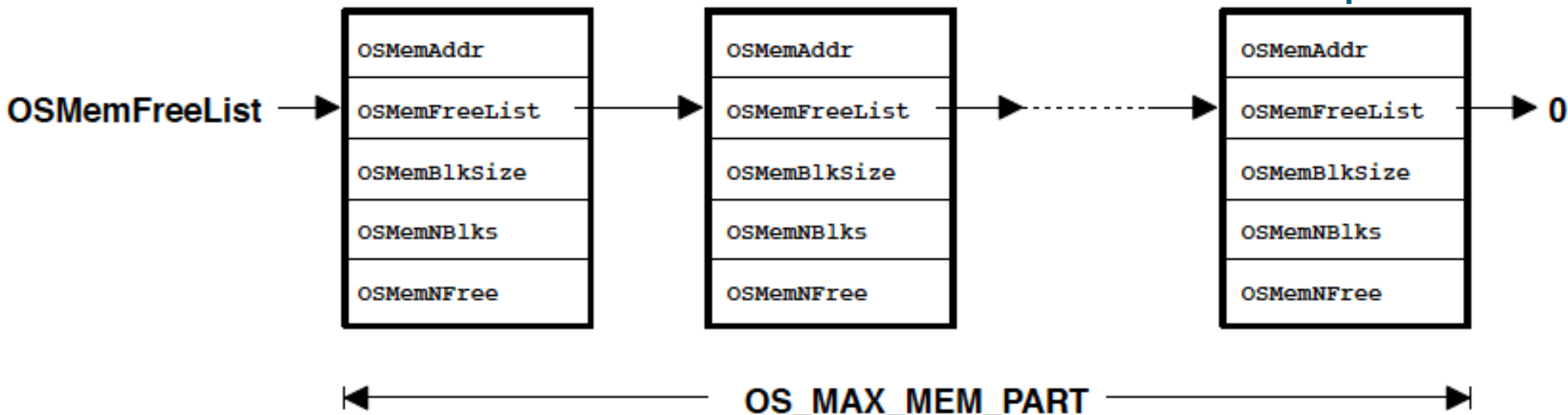
Micro C

Gestión de Memoria

- Los MCB forman una lista encadenada, que se emplea en la asignación y liberación de memoria; cuando se asigna a una tarea se saca de la lista y cuando se libera se incluye nuevamente en la lista

```
typedef struct {  
    void    *OSMemAddr;  
    void    *OSMemFreeList;  
    INT32U  OSMemBlkSize;  
    INT32U  OSMemNBlks;  
    INT32U  OSMemNFree;  
} OS_MEM;
```

OSMemQuery()



Micro C

Gestión de Memoria

- antes de usar una partición debe crearse

`OS_MEM *OSMemCreate (void *addr, INT32U nblks, INT32U blksize, INT8U *err)`

```
OS_MEM *CommTxBuf;  
INT8U CommTxPart[100][32];
```

Partición de 100
bloques de 32
bytes cada uno

```
void main (void)  
{
```

```
    INT8U err;
```

```
    OSInit();
```

```
    ...
```

```
    CommTxBuf = OSMemCreate(CommTxPart, 100, 32, &err);
```

```
    ...
```

```
    OSStart();
```

```
}
```

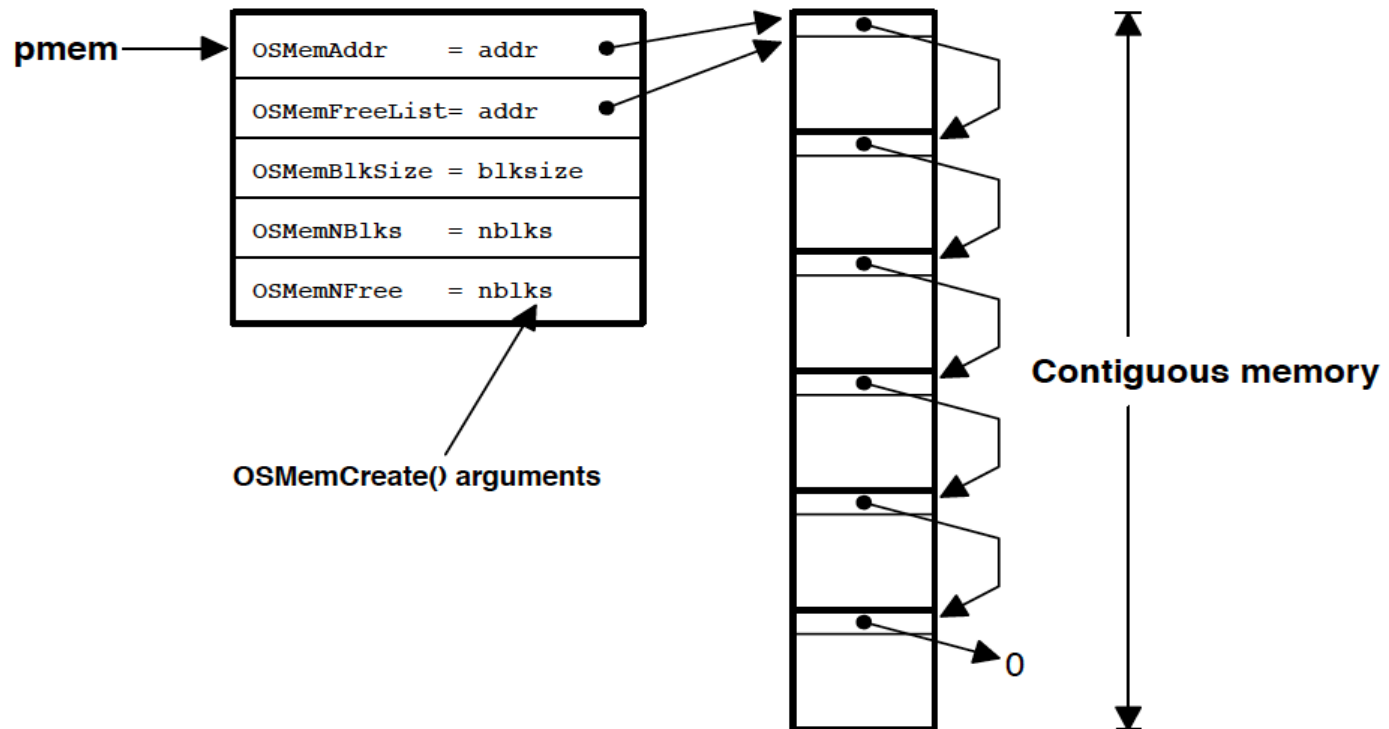
Dirección de la
partición



Micro C

Gestión de Memoria

- Se crean bloques contiguos.
- Cuando pidas un bloque te lo darán (si hay libres) pero no sabes cual.



Micro C

Gestión de Memoria

`void *OSMemGet (OS_MEM *pmem, INT8U *err)`

- devuelve un puntero a un bloque de una partición para su uso (la aplicación debe tener en cuenta el tamaño del bloque)

`INT8U *OSMemPut (OS_MEM *pmem, void *pblk)`

- libera un bloque que ya no se necesita y lo devuelve a la partición (hay que devolverlo a la partición de la que se tomó)

`INT8U *OSMemQuery (OS_MEM *pmem, OS_MEM_DATA *pdata)`

- Actualiza una estructura de datos apuntada por `pdata` con los valores de la partición apuntada por `pmem`, que incluye dirección de la partición, puntero a la lista de bloques libres, tamaño y número de bloques totales, usados y libres

Micro C

Gestión de Memoria

```
#include "includes.h"
#include <stdio.h>

#define TASK_STACK_SIZE    256
#define TASK_ADC_PRIO      5
#define TASK_PROC_PRIO     6
#define QUEUE_SIZE         10
#define TEMP_THRESHOLD     75.0
#define NUM_BLOCKS         10

typedef struct {
    float temperatura;
    INT8U canal;
    float ruido;
} ADC_Data;

OS_STK TaskADCStk[TASK_STACK_SIZE];
OS_STK TaskProcStk[TASK_STACK_SIZE];

OS_EVENT *MsgQueue;
void *QueueStorage[QUEUE_SIZE];
```

```
OS_MEM *MemPart;           // Puntero a la partición
ADC_Data MemPool[NUM_BLOCKS]; // Espacio para los bloques
INT8U MemErr;
```

```
int main(void) {
    OSInit();

    // Crear cola
    MsgQueue = OSQCreate(&QueueStorage[0], QUEUE_SIZE);

    // Crear partición de memoria
    MemPart = OSMemCreate((void *)&MemPool[0], NUM_BLOCKS,
        sizeof(ADC_Data), &MemErr);
    if (MemErr != OS_NO_ERR) {
        printf("Error creando partición de memoria\n");
        return -1;
    }

    OSTaskCreate(TaskADC, NULL, &TaskADCStk[TASK_STACK_SIZE - 1],
        TASK_ADC_PRIO);
    OSTaskCreate(TaskProcessor, NULL, &TaskProcStk[TASK_STACK_SIZE - 1],
        TASK_PROC_PRIO);

    OSStart();
    return 0;
}
```

Micro C

Gestión de Memoria

```
void TaskADC(void *pdata) {
    ADC_Data *adc_msg;
    INT8U err;
    while (1) {
        adc_msg = (ADC_Data *)OSMemGet(MemPart, &err);
        if (err == OS_NO_ERR && adc_msg != NULL) {
            // Simulación de lectura ADC
            adc_msg->temperatura = (rand() % 100);
            adc_msg->canal = rand() % 4;
            adc_msg->ruido = ((float)(rand() % 10))/10.0;

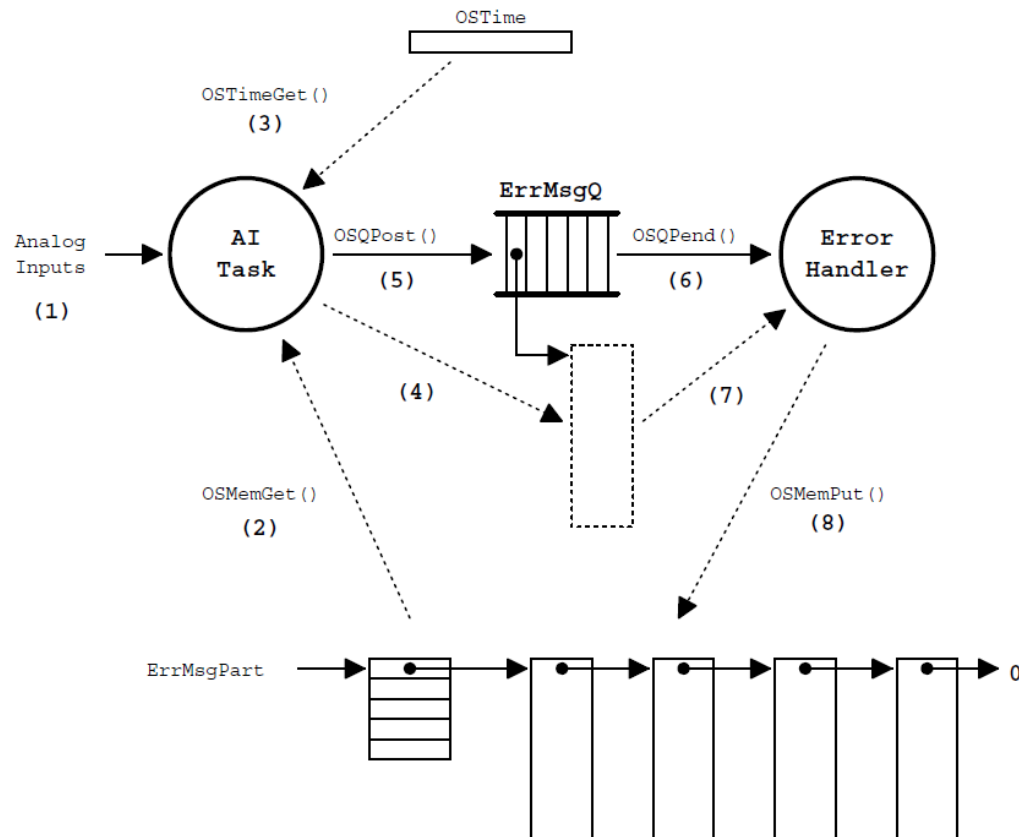
            if (OSQPost(MsgQueue, (void *)adc_msg) ==
OS_NO_ERR) {
                printf("ADC: enviado canal %d, temp %.2f,
ruido %.2f\n",
                    adc_msg->canal, adc_msg->
temperatura, adc_msg->ruido);
            } else {
                // Si la cola está llena, devolver bloque
                OSMemPut(MemPart, adc_msg);
            }
        } else {
            printf("ADC: No hay bloques disponibles\n");
        }
        OSTimeDlyHMSM(0, 0, 1, 0);
    }
}
```

```
void TaskProcessor(void *pdata) {
    ADC_Data *received;
    INT8U err;
    while (1) {
        received = (ADC_Data *)OSQPend(MsgQueue, 0, &err);
        if (err == OS_NO_ERR && received != NULL) {
            float valor_filtrado = received->temperatura -
received->ruido;
            if (valor_filtrado > TEMP_THRESHOLD) {
                printf("ALERTA: Canal %d supera umbral
(%.2f)\n",
                    received->canal, valor_filtrado);
            } else {
                printf("Procesado: Canal %d, Temp %.2f\n",
                    received->canal, valor_filtrado);
            }
            // Devolver bloque a la partición
            OSMemPut(MemPart, received);
        }
    }
}
```

Micro C

Gestión de Memoria

- Otro similar: Cola de errores ADC y memoria de bloques para errores de conversión (canal_ADC,etc)



Micro C

Gestión de Memoria

```

AnalogInputTask()
{
    for (;;) {
        for (all analog inputs to read) {
            Read analog input;                                (1)
            if (analog input exceeds threshold) {
                Get memory block;                               (2)
                Get current system time (in clock ticks);       (3)
                Store the following items in the memory block.  (4)
                    System time (i.e. a time stamp);
                    The channel that exceeded the threshold;
                    An error code;
                    The severity of the error;
                    Etc.
                Post the error message to error queue;         (5)
                    (A pointer to the memory block containing the data)
            }
        }
        Delay task until it's time to sample analog inputs again;
    }
}

```

Micro C

Gestión de Memoria

```
ErrorHandlerTask()
{
    for (;;) {
        Wait for message from error queue;                (6)
        (Gets a pointer to a memory block containing information
         about the error reported)
        Read the message and take action based on error reported; (7)
        Return the memory block to the memory partition;    (8)
    }
}
```

Micro C

Gestión de Memoria

- UCOS no tiene pending de memoria
- Se pueden usar semáforos con el número de bloques.

```
OS_EVENT  *SemaphorePtr;           (1)
OS_MEM     *PartitionPtr;
INT8U      Partition[100][32];
OS_STK     TaskStk[1000];

void main (void)
{
    INT8U err;

    OSInit();                       (2)
    .
    .
    SemaphorePtr = OSSemCreate(100); (3)
    PartitionPtr = OSMemCreate(Partition, 100, 32, &err); (4)
    .
    OSTaskCreate(Task, (void *)0, &TaskStk[999], &err); (5)
    .
    OSStart();                       (6)
}

void Task (void *pdata)
{
    INT8U err;
```


Micro C

Gestión de Memoria

- La tarea que solicita memoria queda pending del semáforo de número de bloques.

```
INT8U *pblock;

for (;;) {
    OSSemPend(SemaphorePtr, 0, &err);           (7)
    pblock = OSMemGet(PartitionPtr, &err);       (8)
    .
    . /* Use the memory block */
    .
    OSMemPut(PartitionPtr, pblock);              (9)
    OSSemPost(SemaphorePtr);                     (10)
}
}
```

Micro C

Hooks

- Los Hooks son funciones que el kernel llama cuando se ejecutan algunas llamadas al sistema.
- Permiten ampliar la funcionalidad del kernel.
- Las más importantes son:
 - `OSTaskSwHook()`, llamada cuando ocurre un cambio de contexto, permite evaluar aspectos estadísticos de cada tarea, o actuar sobre señales de debug.
 - `OSTimeTickHook()` se llama al inicio de `OSTimeTick()` y permite controlar eventos tales como actualización o disparo de timers.
 - `OSTaskCreateHook()` y `OSTaskDelHook()` al crear o eliminar una tarea.
 - `OSTaskIdleHook()` y `OSTaskStatHook()` al ejecutar las tareas IDLE y STAT, respectivamente.

Micro C

Hooks

- OSTaskStatHook()
- Es llamada “cada segundo” por OSTaskStat().
- Permite calcular el tiempo de ejecución de cada tarea y el porcentaje de CPU utilizado.
- Permite saber cuántas veces se ejecuta una tarea.
- No tiene argumentos y no devuelve nada.



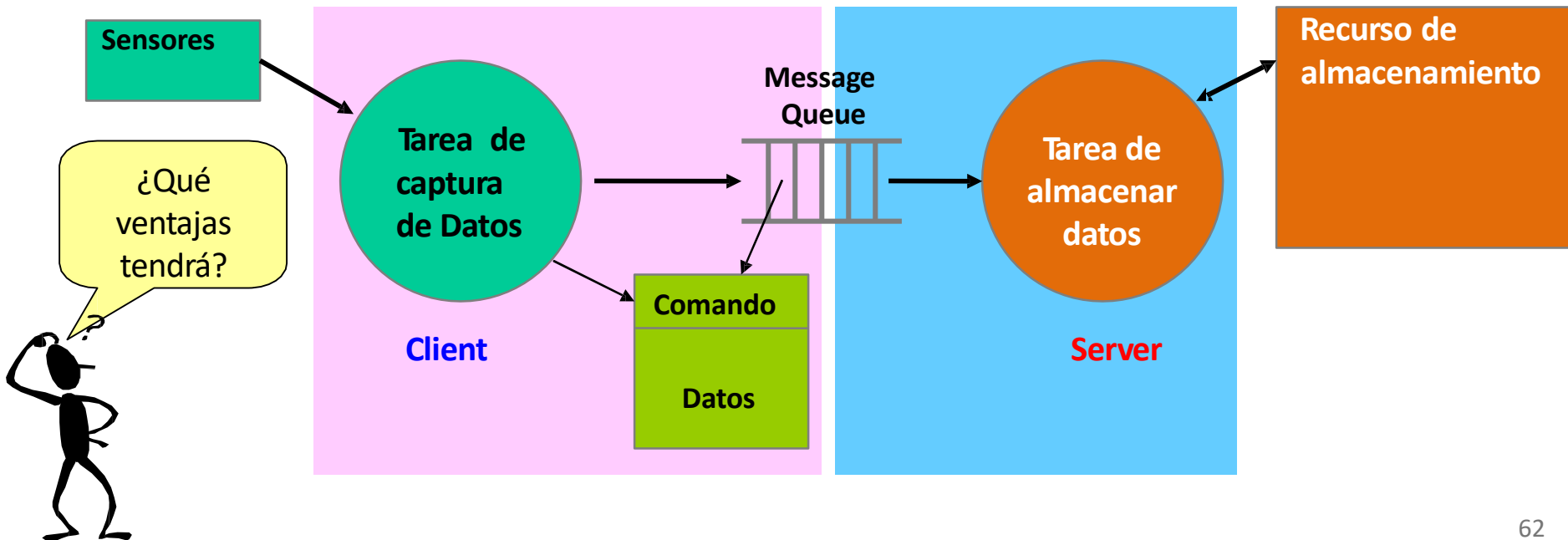
Diseño De aplicaciones con RTOS. Consejos.

División de una aplicación en tareas

- Tratar de detectar actividades que sean atendidas en paralelo
- Identificar tareas cíclicas
- Discriminar las distintas funcionalidades a atender:
 - Interfaz con el operador
 - Barrido de teclado y actualización de display
 - Manejo de mecanismos (impresora, por ejemplo)
 - Medición y procesamiento de datos analógicos (ej: linealización o filtrado)
 - Monitorización y procesamiento de datos discretos (ej: antirebote de contactos)
 - Manejo de errores
 - Contabilización del tiempo (RTC, por ejemplo)
 - Control
 - Comunicaciones, etc.

División de una aplicación en tareas

- Considerar que un *Device Driver* puede necesitar más de una tarea
- Modelar tareas que brinden servicios Servidor/Cliente. Por ejemplo, recursos de almacenamiento masivo.
- Aprovechar los mecanismos de comunicación que brinda el RTOS

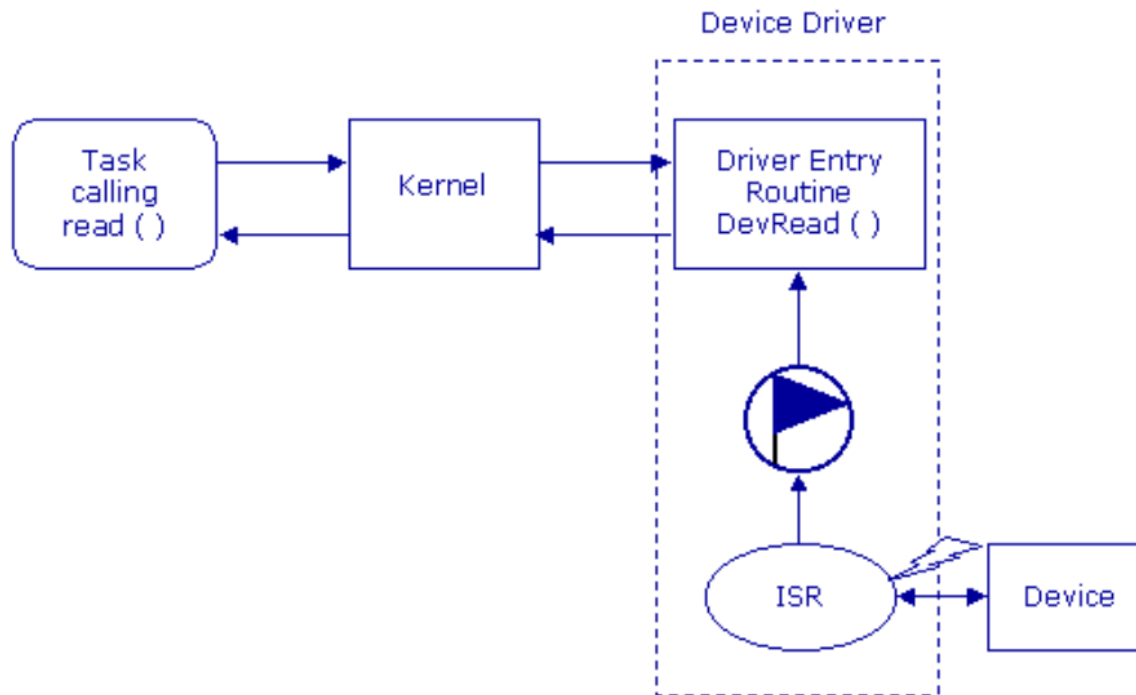


Device Drivers

- *Device Driver: rutinas software que manejan un periférico HW.*
- Un *Device Driver* combina software de inicialización (setup) del periférico, software de lectura y software de escritura.
- Normalmente se debe separar el software “hardware dependent” del resto.
- El dependiente de HW incluye:
 - Rutinas de inicialización del HW (setup) general.
 - Device driver de periféricos.
 - ISR.
- Integrar los Device Drivers con RTOS es necesario.
- Se inicializan con el arranque del RTOS (tarea de `init()`).
- Se especifica si se maneja con ISR o por polling. Habitual por ISR.

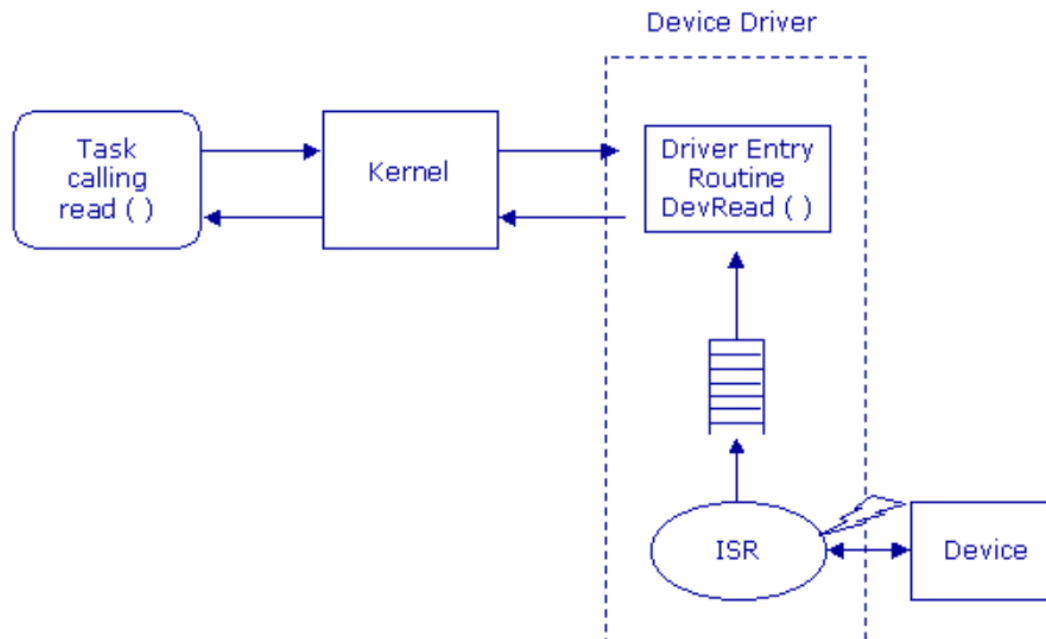
Synchronous Device Drivers

- *La tarea espera que se reciba información desde ISR. Por ejemplo, se bloquea en espera de un semáforo.*



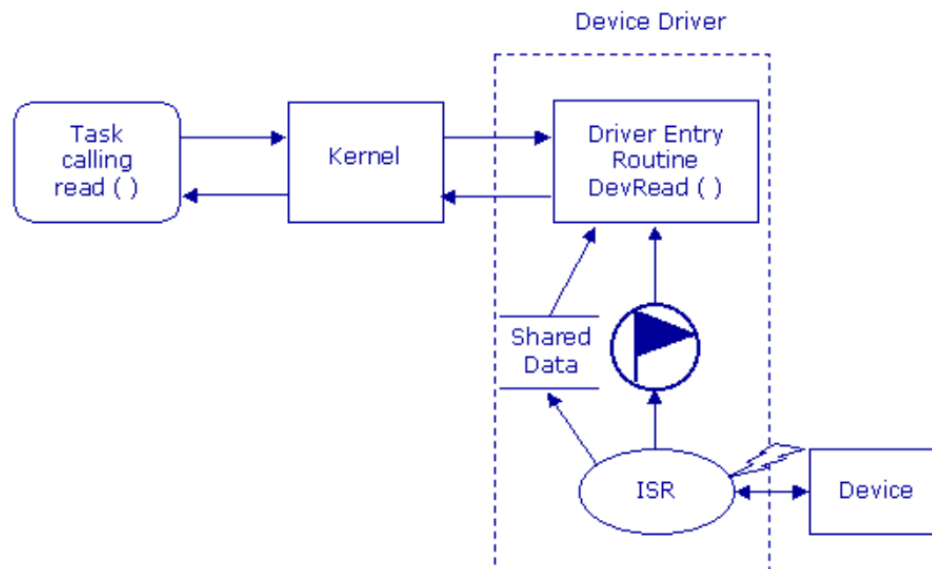
Asynchronous Device Drivers

- *La tarea chequea si hay datos, pero no se bloquea (accept vs pend).*
- *La ISR envía mensajes a cola y la task chequea si hay algo en cola de forma asíncrona.*



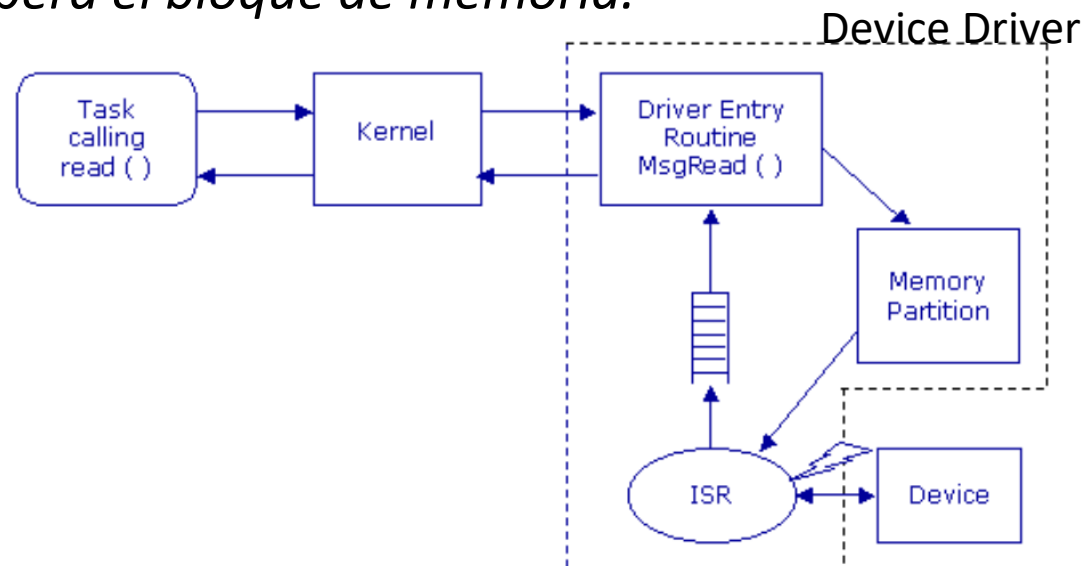
Latest Input Only

- *La tarea necesita solo el último dato obtenido del device. Por ejemplo, la última temperatura leída. Se puede usar un semáforo que controla acceso a memoria compartida (puntero).*
- *¡En uCOS podemos usar una cola LIFO!*



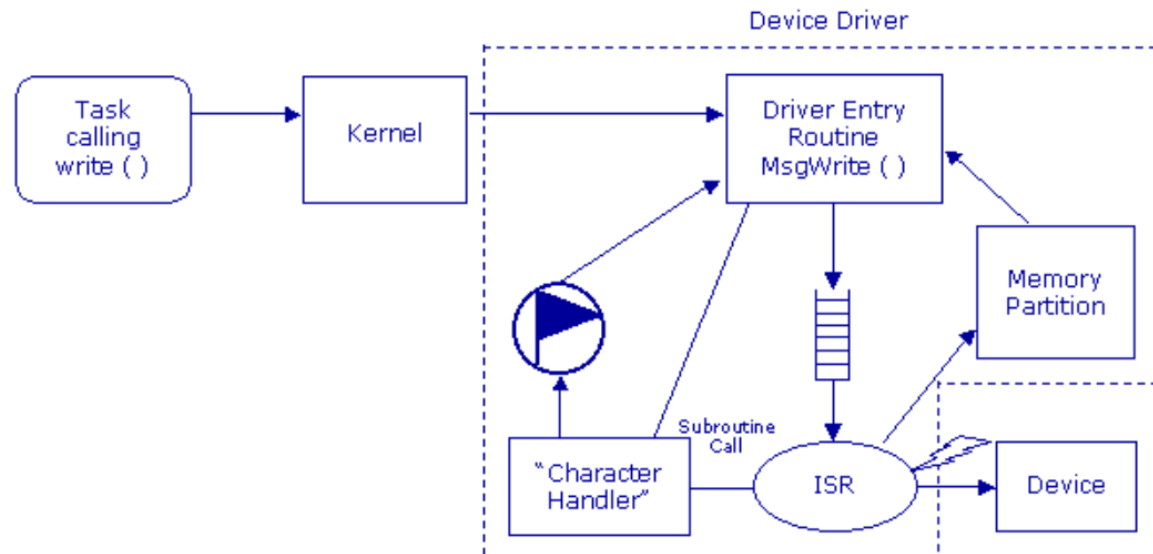
Serial input

- *La ISR recibe datos de periférico (UART), toma una partición de memoria y escribe en ella el mensaje recibido. La ISR además envía puntero a bloque de memoria reservado a una cola.*
- *La tarea se comunica con una rutina que gestiona el device driver (MsgRead()). Esta rutina lee el mensaje de la cola y se lo da a la tarea mientras libera el bloque de memoria.*



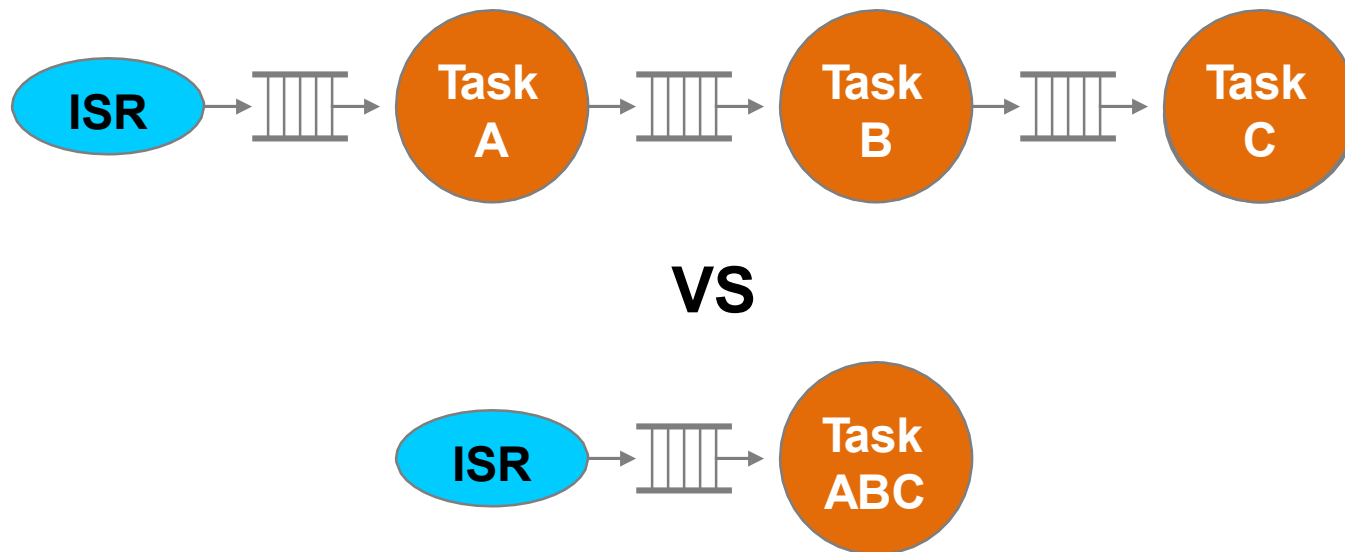
Serial output

- *La tarea escribe mediante una comunicación con `MsgWrite()`, esta rutina obtiene una partición de memoria y el puntero lo pasa a una cola. La ISR toma el puntero y envía los datos de la partición y la libera.*
- *Si la cola está vacía la Isr se lo notifica a `MsgWrite()` vía semáforo.*



División de una aplicación en tareas

- Minimizar la interacción entre tareas
 - Una interdependencia intensa entre tareas suele indicar un mal diseño



División de una aplicación en tareas

- Diseñar tareas complejas y multifuncionales quitan la razón de ser y utilidad de usar un RTOS
- No exagerar en el fraccionamiento de la aplicación
 - Aumenta el overhead generado por el kernel para la conmutación de tareas, el uso de RAM, etc.
- Aprovechar la estructura jerárquica que permite el lenguaje C y realizar descripciones simples de cada tarea
 - Una tarea puede invocar múltiples funciones para hacer su trabajo
- Como regla inicial, una tarea debe requerir mucho más tiempo que el tiempo del cambio de contexto

División de una aplicación en tareas.

Algunos consejos útiles (I)

- Desincronizar operaciones con plazos muy diferentes separándolas en tareas diferentes
 - Ejemplo: no poner en la misma tarea un ajuste de una curva y responder al ABS
 - Poner el ajuste en una tarea de baja prioridad
 - La respuesta al ABS en una tarea de alta prioridad
- Operaciones de interfaz de usuario
 - Si la respuesta al usuario es solo activar un led -> ISR
 - Si el procesamiento es más elaborado (100 ms) -> ISR -> tarea
- Operaciones de “milisegundos”
 - Tareas de alta prioridad
 - Si deben ser más rápidas (μ s) directamente en ISR

División de una aplicación en tareas.

Algunos patrones útiles (II)

- Operaciones acaparadoras de CPU (segundos)
 - Tareas de baja prioridad
- Operaciones de monitorización
 - Tareas que hay que hacer cuando no hay nada más que hacer
 - Tareas de la menor prioridad
- En general, es importante identificar:
 - Cuáles deben cumplir necesariamente exigencias temporales
 - Lazos de Control
 - Ciertos tipos de comunicaciones
 - Alarmas y acciones de emergencia
 - I/O (analógica, discreta...)
 - Cuáles requieren pautas temporales pero toleran ocasionales excesos
 - Cuáles no son críticas en el aspecto temporal, y toleran latencias de atención importante.
 - Interfaz con el operador (teclados, displays, ...)